

## Assignment 2

Student name: *Daniele Mensorio*

---

Course: *Numerical methods for Data Analysis* – Professor: *Prof.essa Rosanna Campagna*  
Due date: *June 17th, 2024*

### Clustering Technique: Functions

Every basic **clustering problem** starts with the same idea: a *Matrix*  $D$ , representing *data points*  $x_{ij}$ ,

$$D = \{x^1, x^2, \dots, x^p\} \quad x^j \in \mathbb{R}^n \quad 1 \leq j \leq p$$

that is partitioned, according to some criterion, into  $k$  *cluster*, groups,  $D_l$ :

$$D_l = \{x^j | j \in I_l\} \quad 1 \leq l \leq k$$

where  $I_l$  is a *index set* corresponding to each cluster  $D_l$  such that it is impossible to find a data point in two different cluster and the sum of all the index set is equal to the number (in this case, the column size) of the data.

$$I_l \cap I_j = \emptyset \quad \forall l \neq j$$

$$\sum_{l=1}^k I_l = \{1, 2, \dots, p\}$$

For each cluster  $D_l$ , it is associated a *characteristic vector*  $c^{(l)}$  that is representative of the cluster. How a vector can represent a set of data points? The answer stays in the minimization of the distance between the data points and the characteristic vector,

$$d(x^j - c^{(l)}) = \min_{1 \leq l \leq k} d(x^j - c^{(l)})$$

such that it is possible to define the *within-cluster coherence* (or within-cluster tightness or within-cluster sum of squares (WCSS)),  $q_l$ , and the *overall coherence* (or overall tightness) of the clustering of data,  $Q$ .

$$q_l = q_l(c^{(l)}) = \sum_{j \in I_l} d(x^j - c^{(l)})$$

$$Q = Q(\Pi, c^{(1)}, c^{(2)}, \dots, c^{(k)}) = \sum_{l=1}^k q_l(c^{(l)})$$

where  $\Pi$  defines the *partitioning* of the data.

$$\Pi = \{I_1, I_2, \dots, I_k\}$$

At this point, the aim is to find a criterion that:

- divides, at best, the data points  $x_{ij}$  into  $k$  clusters with properties listed above;
- $\forall l, 1 \leq l \leq k$ , finds the best characteristic vector  $c^{(l)}$ : the best is the one that minimizes the WCSS,  $q_l$ , and the overall coherence,  $Q$ ;
- as said in the previous point, minimizes the within-cluster coherence,  $q_l$ , and the overall tightness,  $Q$ , of the clustering of the data,  $\Pi$ .

The most commonly used clustering algorithms, that solve the problem listed above, are the *K-means* and the *k-medoids* one.

**K-means Algorithm.** The *k-means* algorithm takes on values, as column of the data  $D$ , only vector: so, the distance measure between the column vector of the data,  $x^j$ , and the characteristic vector,  $c^{(l)}$ , the euclidean distance between vector column of the same size.

$$d(x^j - c^{(l)}) = \|x^j - c^{(l)}\|_2$$

As consequence, the definition of the within-cluster coherence,  $q_l$ , and the overall coherence,  $Q$ , are, as follow:

$$q_l = q_l(c^{(l)}) = \sum_{j \in I_l} \|x^j - c^{(l)}\|_2$$

$$Q = Q(\Pi, c^{(1)}, c^{(2)}, \dots, c^{(k)}) = \sum_{l=1}^k q_l$$

In this case, the minimization problem (minimize the overall coherence,  $Q$ ) is reduced to put the gradient of the overall tightness equal to zero such that to find the minimum value, that is the best characteristic vector,  $c^{(l)}$ . The result of this minimization problem is that the characteristic vector is a *centroid* that correspond to the mean of each cluster.

$$\nabla_{c^{(l)}}(Q(\Pi, c^{(1)}, c^{(2)}, \dots, c^{(k)}))$$

$$c^{(l)} = \frac{1}{|D_l|} \sum_{j \in I_l} x^j$$

So, the centroid, the mean of each cluster, is the vector that, at the best, represents each group of data and that minimizes the overall coherence of the clustering. If we've use a *1-norm* or other kind of distance measure, we would not have reached to find the centroid as the characteristic vector of the algorithm. My k-means algorithm can be found in the Appendix and is an application of the *K-means* of the **Lloyd's Algorithm**.

The assignment step in the algorithm has a geometrical meaning: it produces a Voronoi tessellation. A **Voronoi tessellation** is made in such a way that each cluster is separated by hyper-planes such that each hyper-plane is equidistant from two centroids. Voronoi sets are a space of dimension  $\mathbb{R}^{n-1}$  because it creates  $n - 1$  clusters divided by hyper-plane.

**Drawbacks:** The 'limitations' of the *k-means* algorithm are:

- even if it is a robust clustering algorithm, it is very sensible of the outliers;

- the use of euclidean norm as distance measure between data points and centroids since euclidean norm can be used only on vectors, excluding a great variety of data formats, like polar coordinates, binary data, and so on.
- When there is no cluster structure in the data, the *k-means* may not provide a reliable tool to classify the clusters.

***K-medoids Algorithm.*** All the drawbacks described above are all limitations that the *k-medoids* algorithm has overcome. First of all, regarding the distance measure, in the *k-medoids* algorithm, the distance between data points in the distance matrix and the distance between data points and the characteristic vectors (the medoids) can be measured in any way possible: if there exist a distance, or dissimilarity, measure between items, a *k-medoids* clustering method can be applied.

The *k-medoids* algorithm applied is an improvement of the older one, the so-called ***Partitioning Around Medoids (PAM)*** algorithm. My algorithm, that is an application of the mentioned one, can be found in the Appendix .

An important feature of the *k-medoids* technique is that the characteristic vector,  $c^{(l)}$ , belongs to a data point inside the cluster; differently from the *k-means* one, in which the centroid could not be a point of the cluster and that could be an issue in some situation: i.e., the case in which the data are distributed, approximately, as a circle, the centroid would be located in the middle, close to the center of the circle. This issue leads to the centroid to not be a good representative of the cluster.

### Iris Dataset

The data file **IrisDataAnnotated.mat** consists of a data matrices  $X$  and  $I$ . The matrix  $X$  consists of 150 vectors,  $X \in \mathbb{R}^{4 \times 150}$ , each one having four components. The data corresponds to measurements of certain dimensions in three species of flowers, *Iris setosa*, *Iris versicolor*, and *Iris virginica*, and the components of the data vectors have the following attributes:

$$x^j = \begin{bmatrix} \text{sepal length in cm} \\ \text{sepal width in cm} \\ \text{petal length in cm} \\ \text{petal width in cm} \end{bmatrix} \in \mathbb{R}^4, \quad 1 \leq j \leq 150$$

The data matrix  $I$ ,  $I \in \mathbb{R}^{1 \times 150}$ , is the label matrix in which to each column is assigned an iris species between the three of the flowers:

$$1 = \text{Iris Setosa} \quad 2 = \text{Iris Versicolor} \quad 3 = \text{Iris Virginica}$$

**K-means.** Before diving into the clustering techniques, my approach is to compute a dimensional reduction, using Principal Component Analysis (PCA), on centering data matrix,  $X_c = X - \bar{X}$ , for the only goal of better visualizing the result since we have already studied that the first Principal Component (PCs) (in the iris data-set, 2 – 3, depending on the choice of a two- or three-dimensional visualisation) are the ones that maximize the spread of the data. So, we compute the SVD of the centering data points,  $X_c = UDV'$ , for then computing the Principal Components (PCs),  $Z$ .

Our aim is to apply my *k-means* (Lloyd's) sub-routine function to the Iris Matrix,  $X$ :

1. First of all, fix  $k$ , the number of clusters (in this case,  $k = 3$ ) and a positive tolerance,  $\tau > 0$  (for this example,  $\tau = 1.0e^{-100}$ ), used as criterion to stop the for-loop iteration;
2. Then, the **initialisation step** in which it is defined a starting partition,  $\Pi_0$ , and the counting is started (it is set  $t = 0$ ): In this case, the initial initialisation is to pick random seed vectors ( $\{c_0^{(1)}, c_0^{(2)}, c_0^{(3)} | c_0^l \in \mathbb{R}^4 | 1 \leq l \leq 3\}$ ) (see Table 1) and divide the data into cluster of , approximately, the same sizes. This part can be clearly seen in figure 1a, for a 2D representation, and in figure 1b, for a three-dimensional one;

| Iris Species              |        |            |         |
|---------------------------|--------|------------|---------|
|                           | Setosa | Versicolor | Virgina |
| <b>Sepal length in cm</b> | 4.50   | 7.90       | 6.10    |
| <b>Sepal width in cm</b>  | 2.30   | 3.80       | 3.00    |
| <b>Petal length in cm</b> | 1.30   | 6.40       | 4.60    |
| <b>Petal width in cm</b>  | 0.30   | 2.00       | 1.40    |

Table 1: Initial seed vector  
Iris Dataset

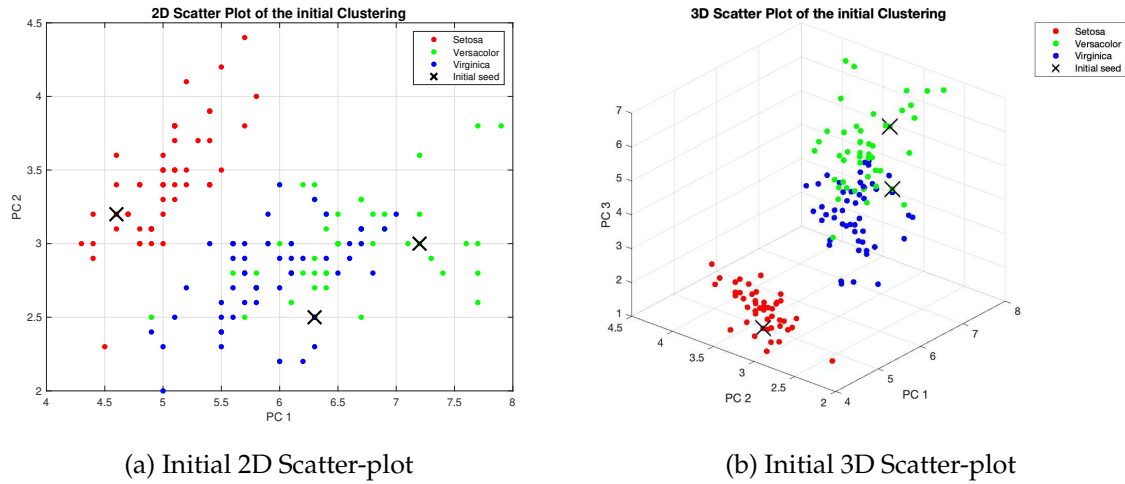


Figure 1: a-b) 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the initial seed vectors, randomly chosen.

### 3. Iteration step:

- (a) **Updating step**, in which it is computed the centroid,  $c_t^{(l)}$ , of the partition,  $\Pi_t$ :

$$c_t^{(l)} = \frac{1}{|D_l|} \sum_{j \in I_l} x^j$$

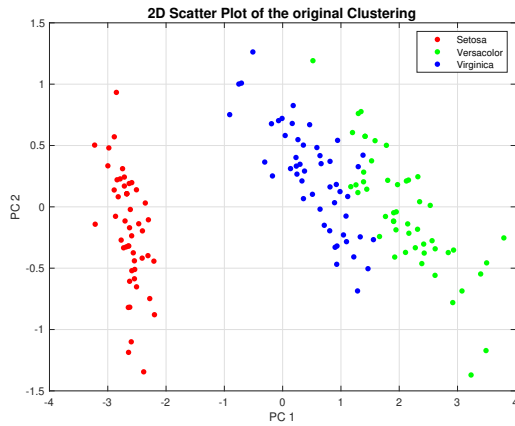
- (b) **Assignment step**:  $\forall x^j, \quad 1 \leq j \leq 150$ , it is computed the minimum distance from the centroids,  $c_t^{(l)}$ , using the euclidean norm (the final centroids, result of my algorithm, can be found at table 2)

$$d(x^j - c_t^{(l)}) = \|x^j - c_t^{(l)}\|_2$$

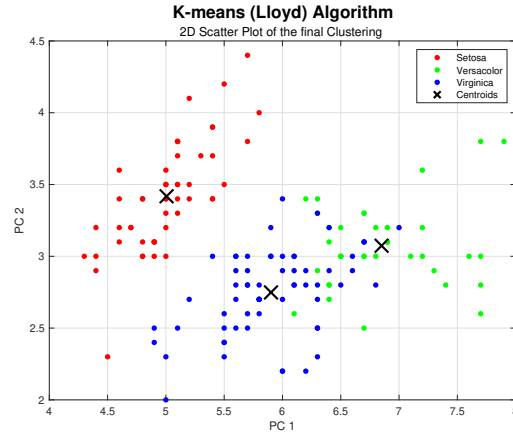
and assign each data points,  $x^j$ , to the corresponding (nearest) cluster. This process defines a new partition,  $\Pi_{t+1}$ . (The new partition, as the consequent final clustering of the Iris data) could be seen at figure 2b, for a bi-dimensional visualisation, and at figure 3a, for a 3D one).

| Iris Species       |        |            |         |
|--------------------|--------|------------|---------|
|                    | Setosa | Versicolor | Virgina |
| Sepal length in cm | 5.01   | 6.85       | 5.90    |
| Sepal width in cm  | 3.42   | 3.07       | 2.75    |
| Petal length in cm | 1.46   | 5.74       | 4.39    |
| Petal width in cm  | 0.24   | 2.07       | 1.43    |

Table 2: Final centroid vector  
Iris Dataset



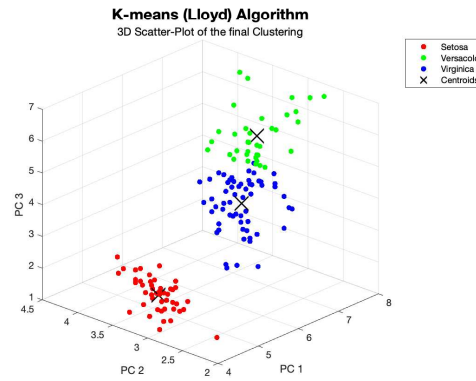
(a) Real 2D Scatter-plot



(b) Final 2D Scatter-plot

The figure 2a is a 2D scatter-plot of the first two principal components (PCs) of the iris data matrix  $X$ , grouped by the real partitioning into cluster, according to the  $I$  column vector, contained into the Matlab data file `IrisDataAnnotated.mat`.

Figure 2: a) 2D scatter-plot of the original matrix according to the  $I$  column vector. b-c) The 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the final clustering results.



(a) Final 3D Scatter-plot

4. If our criterion, in this case, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , is lower the tolerance,  $\tau$ , fixed before, the algorithm can stop; otherwise, the counter continues (it is set  $t = t + 1$ ), and the algorithm goes to the step 3.

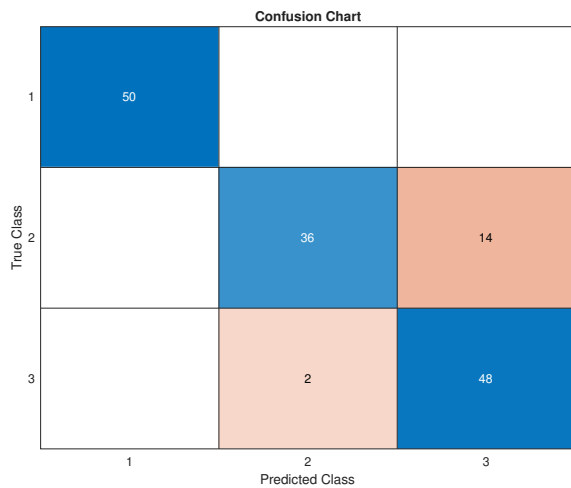
**If**

$$|Q(\Pi_t) - Q(\Pi_{t+1})| < \tau \quad \text{stop}$$

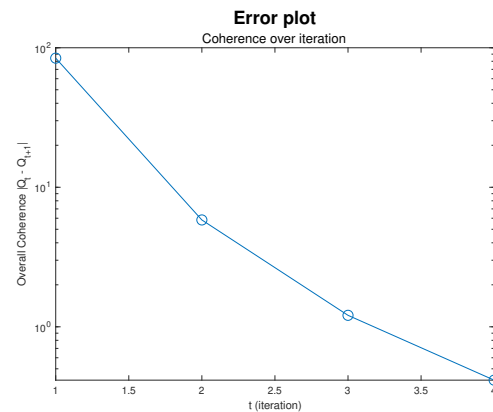
**else**

$$t = t + 1, \quad \text{go to step 3.}$$

The progress of the criterion, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , could be seen at figure 3c. It can be notice that after 4 iteration, the criterion reach 0, meaning that the overall coherence of the partitioning are equal  $Q(\Pi_t) = Q(\Pi_{t+1})$



(b) Confusion Chart



(c) Overall Coherence plot

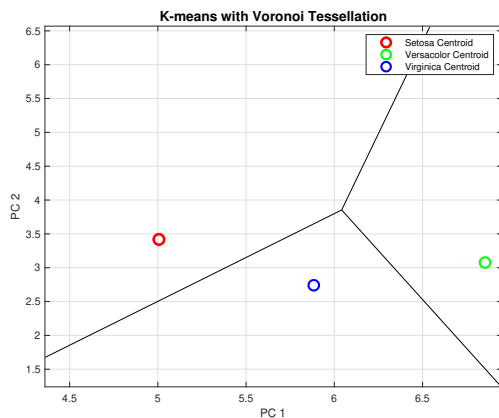
Figure 3: a) Confusion Chart derived from the comparison of the  $I_{assign}$  (produced by my  $k$ -means clustering) and the  $I$  column vectors. b) Plot of the difference between the overall coherence of preceding clustering,  $Q_t$ , and of the last one,  $Q_{t+1}$ ,  $|Q_t - Q_{t+1}|$ , over the iteration,  $t$ .

The figure 3b is a visualisation of the confusion matrix (indeed, the confusion chart) in which can be seen the errors that my algorithm are made, comparing the  $I$  and my  $I_{assign}$  column vector: In this case,

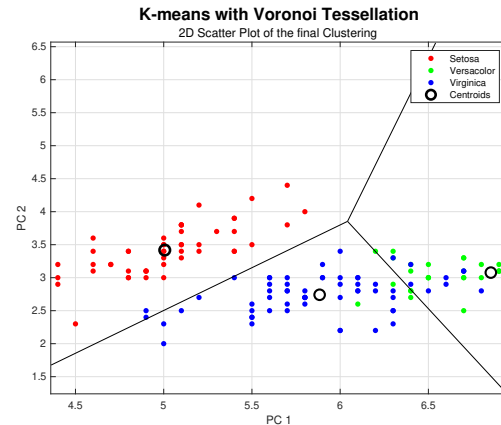
- the  $k$ -means found all the points associated to the first cluster, 50/50, with an error of 0%.
- for the second cluster, the algorithm found 36/50, 36 points belong to the second cluster out of the total 50, incorrectly assigning the last 14 to third cluster, producing an error of 28%.
- the clustering technique found 48 points belonging to the third cluster, erroneously assigning 2 vectors tot the second cluster, emitting an error of 4%.

The assignment step of the  $k$ -means (Lloyd's) Algorithm has a geometrical meaning: it produces, from a geometric view point, a **Voronoi tessellation**. A Voronoi tessellation is a space of  $\mathbb{R}^{n-1}$  in which each cluster is separated by hyper-planes such that each hyper-plane is equidistant from two centroids. The sets is of size  $n - 1$  because it creates  $n - 1$  clusters divided by hyper-plane. A proof of this theoretical foundation is the Voronoi tessellation (see figure 4a), produced by my  $k$ -means algorithm (on Matlab) with a pre-fixed number of cluster  $k = 3$ , and the same with also the 2D scatter-plot of the principal components (PCs) of the data, grouped by the predicted classes (see figure 4b).

For a better visualisation of geometrical implication of the assignment step, the Voronoi tessellation, the figure ?? and the consequent one 5b are representation of the Voronoi sets when the pre-fixed number of cluster  $k = 10$ .

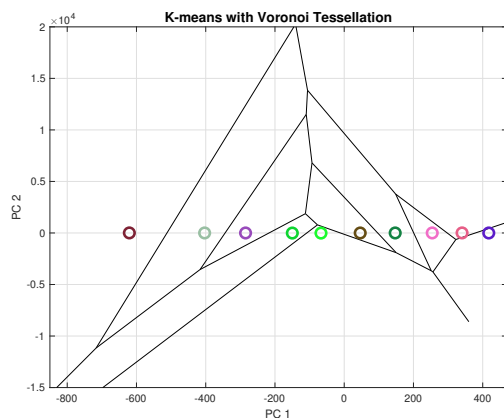


(a) Voronoi tessellation

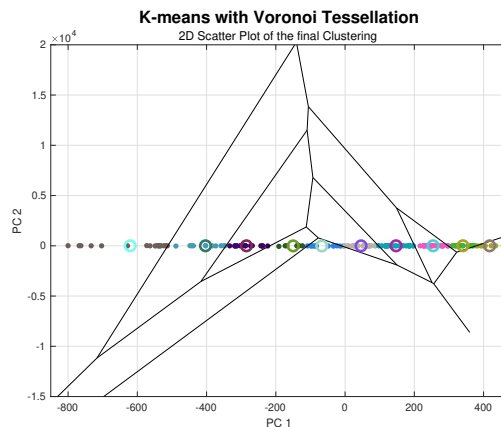


(b) 2D Scatter-plot with Voronoi sets

Figure 4: a) Voronoi tessellation of the final centroids obtained from the *k-means* clustering technique with fixed  $k = 3$  cluster. b) 2D scatter-plot of the final *k-means* clustering of the data with Voronoi sets with fixed  $k = 3$  cluster.



(a) Voronoi tessellation



(b) 2D Scatter-plot with Voronoi sets

Figure 5: a) Voronoi tessellation of the final centroids obtained from the *k-means* clustering technique with fixed  $k = 10$  cluster. b) 2D scatter-plot of the final *k-means* clustering of the data with fixed  $k = 10$  cluster with Voronoi sets.

### *K-medoids.*

**1-norm** Before diving into the clustering techniques, my approach is to compute a dimensional reduction, using Principal Component Analysis (PCA), on centering data matrix,  $X_c = X - \bar{X}$ , for the only goal of better visualizing the result since we have already studied that the first Principal Component (PCs) (in the iris data-set, 2 – 3, depending on the choice of a two- or three-dimensional visualisation) are the ones that maximize the spread of the data. So, we compute the SVD of the centering data points,  $X_c = UDV'$ , for then computing the Principal Components (PCs),  $Z$ .



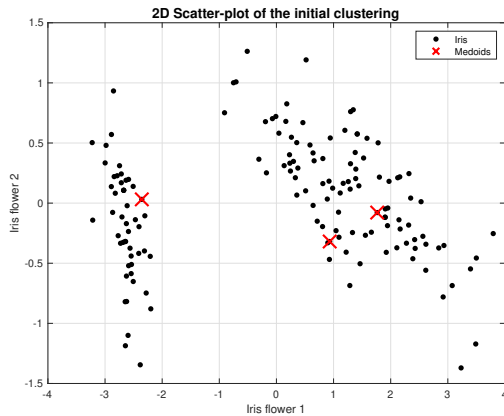
Our aim is to apply my *k-medoids* (PAM (Partitioning Around Medoids)) sub-routine function to the Iris Matrix,  $X$ :

1. First of all, fix  $k$ , the number of clusters (in this case,  $k = 3$ ) and a positive tolerance,  $\tau > 0$  (for this example,  $\tau = 1.0e^{-100}$ ), used as criterion to stop the for-loop iteration;
2. Then, the **initialisation step** in which it is chosen the initial set of medoids and the counting is started (it is set  $t = 0$ ): In this case, the initial characteristic vectors are picked randomly (with the difference that, with the respect of *k-means*, they belong to one of the data point of the cluster,  $c_0^{(l)} \in D_l$ ) ( $\{c_0^{(1)}, c_0^{(2)}, c_0^{(3)} | c_0^{(l)} \in \mathbb{R}^4 | 1 \leq l \leq 3\}$ ) (see Table 3). This part can be clearly seen in figure 6a, for a 2D representation, and in figure 6b, for a three-dimensional one;

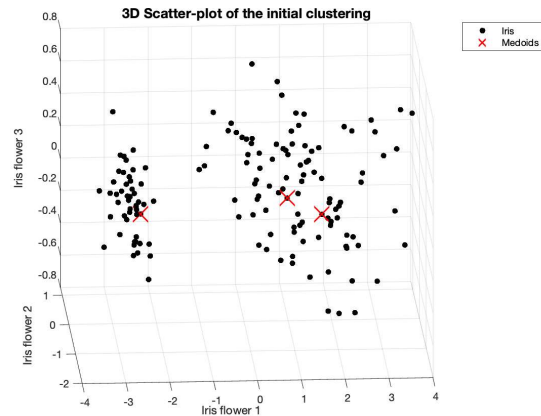
|                    | Iris Species |            |           |
|--------------------|--------------|------------|-----------|
|                    | Setosa       | Versicolor | Virginica |
| Sepal length in cm | -2.684       | 2.165      | -0.751    |
| Sepal width in cm  | -0.327       | -0.215     | 1.001     |
| Petal length in cm | 0.022        | -0.033     | -0.016    |
| Petal width in cm  | 0.001        | 0.163      | -0.017    |

Table 3: Initial medoids

## Iris Dataset



(a) Initial 2D Scatter-plot



(b) Initial 3D Scatter-plot

Figure 6: a-b) 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the initial seed of the characteristic vectors, the medoids,  $c^{(l)}$ , randomly chosen.

### 3. Iteration step:

- (a) **Assignment step:**  $\forall x^j, \quad 1 \leq j \leq 150$ , it is computed the minimum distance between each data points and the initial medoids, randomly chosen, (in this

case, the distance measure used is the 1norm)

$$d(x^j, c_t^{(l)}) = \min_{1 \leq l \leq k} d(x^j, c_t^{(l)})$$

$$d(x^j, c_t^{(l)}) = \|x^j - c_t^{(l)}\|_1$$

and assign each data points,  $x^j$ , to the corresponding (nearest) cluster. This process defines a partition,  $\Pi_t$ , for which it is possible to compute the within-cluster tightness,  $q_l(c_t^{(l)})$ , and the overall coherence,  $Q(\Pi_t)$ .

- (b) **Updating step**, in which for each cluster,  $\forall D_l$ , it is made a local distance matrix consisting of the distance between the data points of the corresponding cluster. So, it is used this matrix to interchange the medoids,  $c_t^{(l)}$ , with each data points of that cluster,  $x^j$ , in order to find the point that minimizes the within-cluster coherence: that it is an easy task since we need to find the minimum sum of the points of each column, the minimum one is the chosen. Then, the within-cluster tightness is recomputed using as medoids the data point chosen,  $q_l(x^j)$ .

If the within-cluster coherence of the data point,  $q_l(x^j)$ , is minor than the within-cluster tightness of the medoid,  $c_t^{(l)}$ , then the data point is interchange with the medoid and this applied also with the within-cluster coherence.

**If**

$$q_l(x^j) < q_l(c_t^{(l)})$$

**then**

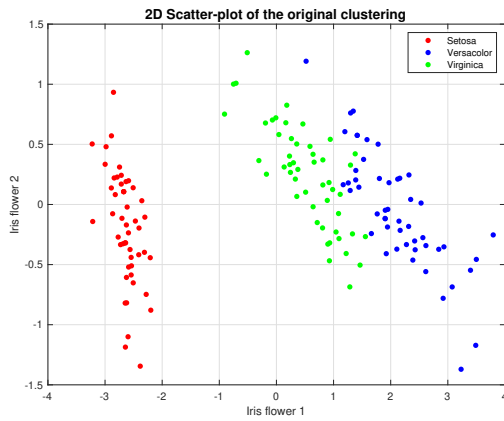
$$c_t^{(l)} = x^j, \quad q_l(c_t^{(l)}) = q_l(x^j)$$

This process produces the final medoids, that can be seen, numerically, in the table 4 and, visually, on the figure 7b and 8a, where it is also possible to see the final clustering of the data into three cluster in a bi-dimensional view i the first figure and in a three- one in the second one.

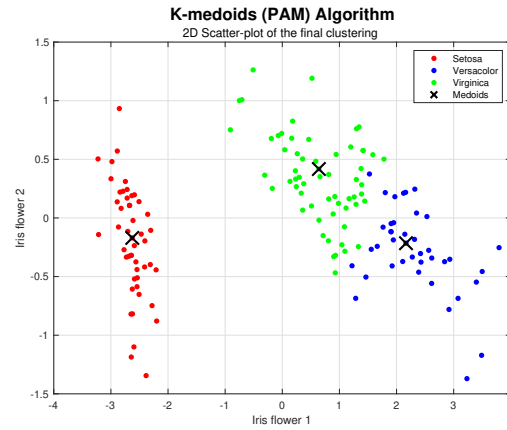
| Iris Species              |        |            |           |
|---------------------------|--------|------------|-----------|
|                           | Setosa | Versacolor | Virginica |
| <b>Sepal length in cm</b> | -2.626 | 1.764      | 0.356     |
| <b>Sepal width in cm</b>  | -0.170 | -0.079     | 0.503     |
| <b>Petal length in cm</b> | 0.016  | -0.131     | -0.018    |
| <b>Petal width in cm</b>  | -0.046 | 0.136      | -0.098    |

Table 4: Final medoids

Iris Dataset



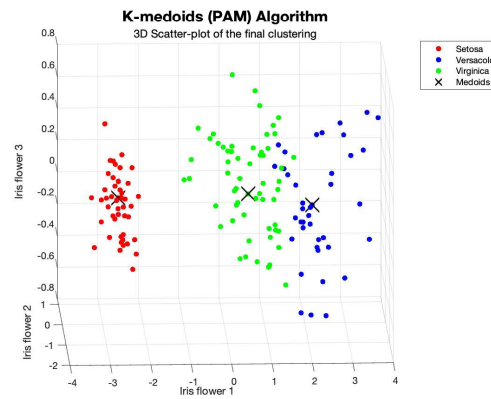
(a) Real 2D Scatter-plot



(b) Final 2D Scatter-plot

The figure 7a is a 2D scatter-plot of the first two principal components (PCs) of the iris data matrix  $X$ , grouped by the real partitioning into cluster, according to the  $I$  column vector, contained into the Matlab data file `IrisDataAnnotated.mat`.

Figure 7: a) 2D scatter-plot of the original matrix according to the  $I$  column vector. b-c) The 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the final clustering results.



(a) Final 3D Scatter-plot

4. If our criterion, in this case, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , is lower the tolerance,  $\tau$ , fixed before, the algorithm can stop; otherwise, the counter continues (it is set  $t = t + 1$ ), and the algorithm goes to the step 3.

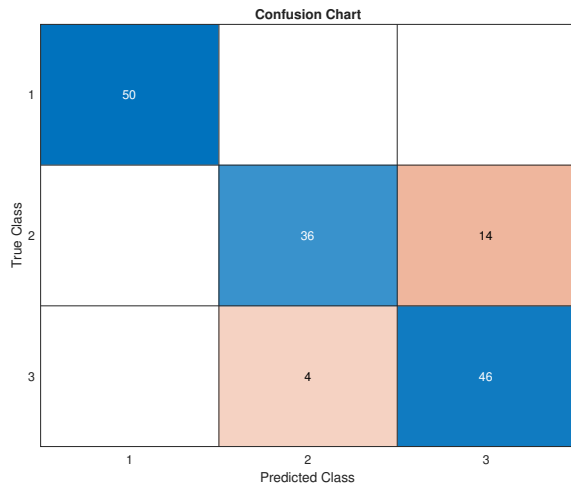
**If**

$$|Q(\Pi_t) - Q(\Pi_{t+1})| < \tau \quad \text{stop}$$

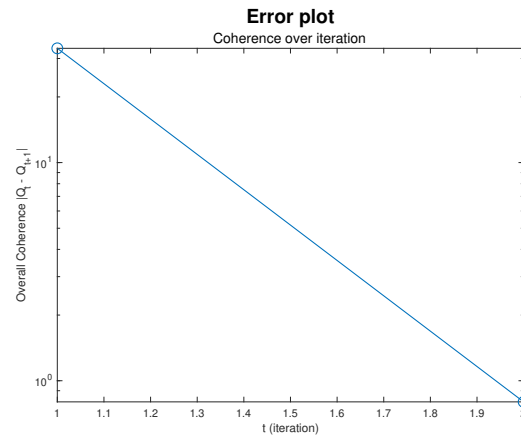
**else**

$$t = t + 1, \quad \text{go to step 3.}$$

The progress of the criterion, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , could be seen at figure 8c. It can be notice that after 2 iteration, the criterion reach 0, meaning that the overall coherence of the partitioning are equal  $Q(\Pi_t) = Q(\Pi_{t+1})$



(b) Confusion Chart



(c) Overall Coherence plot

Figure 8: a) Confusion Chart derived from the comparison of the  $I_{assign}$  (produced by my  $k$ -medoids clustering) and the  $I$  column vectors. b) Plot of the difference between the overall coherence of the last one,  $Q_{t+1}$ ,  $|Q_t - Q_{t+1}|$ , over the iteration,  $t$ .

The figure 8c is a visualisation of the confusion matrix (indeed, the confusion chart) in which can be seen the errors that my algorithm are made, comparing the  $I$  and my  $I_{assign}$  column vector: In this case,

- the  $k$ -means found all the points associated to the first cluster, 50/50, with an error of 0%.
- for the second cluster, the algorithm found 36/50, 36 points belong to the second cluster out of the total 50, incorrectly assigning the last 14 to third cluster, producing an error of 28%.
- the clustering technique found 46 points belonging to the third cluster, erroneously assigning 4 vectors tot the second cluster, emitting an error of 8%.

**2-norm** Before diving into the clustering techniques, my approach is to compute a dimensional reduction, using Principal Component Analysis (PCA), on centering data matrix,  $X_c = X - \bar{X}$ , for the only goal of better visualizing the result since we have already studied that the first Principal Component (PCs) (in the iris data-set, 2 – 3, depending on the choice of a two- or three-dimensional visualisation) are the ones that maximize the spread of the data. So, we compute the SVD of the centering data points,  $X_c = UDV'$ , for then computing the Principal Components (PCs),  $Z$ .

Our aim is to apply my  $k$ -medoids (PAM (Partitioning Around Medoids)) sub-routine function to the Iris Matrix,  $X$ :

1. First of all, fix  $k$ , the number of clusters (in this case,  $k = 3$ ) and a positive tolerance,  $\tau > 0$  (for this example,  $\tau = 1.0e^{-100}$ ), used as criterion to stop the for-loop iteration;
2. Then, the **initialisation step** in which it is chosen the initial set of medoids and the counting is started (it is set  $t = 0$ ): In this case, the initial characteristic vectors are picked randomly (with the difference that, with the respect of  $k$ -means,

they belong to one of the data point of the cluster,  $c_0^{(l)} \in D_l$  ( $\{c_0^{(1)}, c_0^{(2)}, c_0^{(3)} | c_0^{(l)} \in \mathbb{R}^4 | 1 \leq l \leq 3\}$ ) (see Table 5). This part can be clearly seen in figure 9a, for a 2D representation, and in figure 9b, for a three-dimensional one;

| Iris Species       |        |            |         |
|--------------------|--------|------------|---------|
|                    | Setosa | Versicolor | Virgina |
| Sepal length in cm | -2.684 | 2.165      | -0.751  |
| Sepal width in cm  | -0.327 | -0.215     | 1.001   |
| Petal length in cm | 0.022  | -0.033     | -0.016  |
| Petal width in cm  | 0.001  | 0.163      | -0.017  |

Table 5: Initial medoids

### Iris Dataset

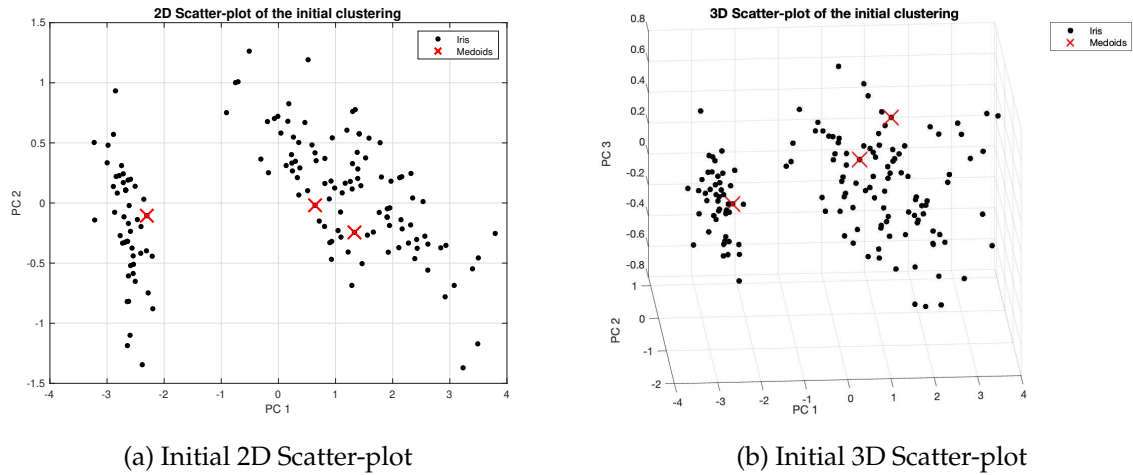


Figure 9: a-b) 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the initial seed of the characteristic vectors, the medoids,  $c^{(l)}$ , randomly chosen.

### 3. Iteration step:

- (a) **Assignment step:**  $\forall x^j, 1 \leq j \leq 150$ , it is computed the minimum distance between each data points and the initial medoids, randomly chosen, (in this case, the distance measure used is the *2norm*)

$$d(x^j, c_t^{(l)}) = \min_{1 \leq l \leq k} d(x^j, c_t^{(l)})$$

$$d(x^j, c_t^{(l)}) = \|x^j - c_t^{(l)}\|_2$$

and assign each data points,  $x^j$ , to the corresponding (nearest) cluster. This process defines a partition,  $\Pi_t$ , for which it is possible to compute the within-cluster tightness,  $q_l(c_t^{(l)})$ , and the overall coherence,  $Q(\Pi_t)$ .

- (b) **Updating step**, in which for each cluster,  $\forall D_l$ , it is made a local distance matrix consisting of the distance between the data points of the corresponding cluster. So, it is used this matrix to interchange the medoids,  $c_t^{(l)}$ , with each data points of that cluster,  $x^j$ , in order to find the point that minimizes the within-cluster coherence: that it is an easy task since we need to find the minimum sum of the points of each column, the minimum one is the chosen. Then, the within-cluster tightness is recomputed using as medoids the data point chosen,  $q_l(x^j)$ .

If the within-cluster coherence of the data point,  $q_l(x^j)$ , is minor than the within-cluster tightness of the medoid,  $c_t^{(l)}$ , then the data point is interchange with the medoid and this applied also with the within-cluster coherence.

**If**

$$q_l(x^j) < q_l(c_t^{(l)})$$

**then**

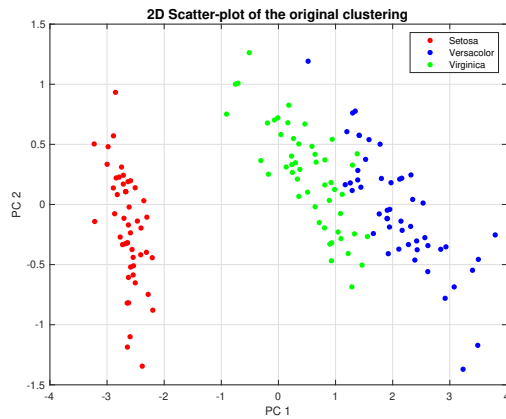
$$c_t^{(l)} = x^j, \quad q_l(c_t^{(l)}) = q_l(x^j)$$

This process produces the final medoids, that can be seen, numerically, in the table 6 and, visually, on the figure 10b and 11a, where it is also possible to see the final clustering of the data into three cluster in a bi-dimensional view i the first figure and in a three- one in the second one.

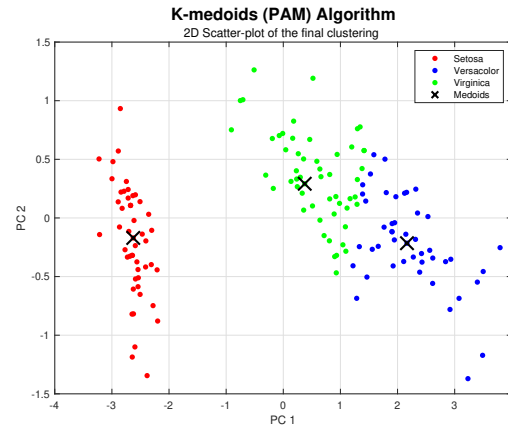
|                           | Iris Species |            |           |
|---------------------------|--------------|------------|-----------|
|                           | Setosa       | Versicolor | Virginica |
| <b>Sepal length in cm</b> | -2.626       | 1.764      | 0.298     |
| <b>Sepal width in cm</b>  | -0.170       | -0.079     | 0.347     |
| <b>Petal length in cm</b> | 0.016        | -0.131     | -0.012    |
| <b>Petal width in cm</b>  | -0.046       | 0.136      | -0.051    |

Table 6: Final medoids

Iris Dataset



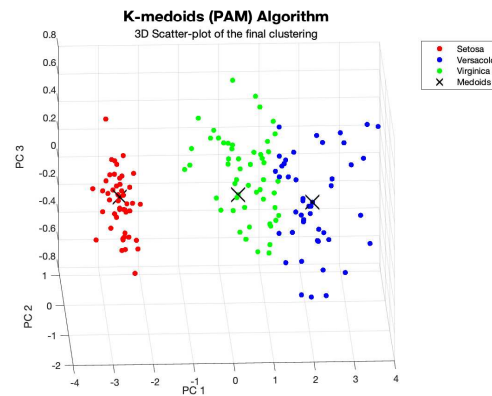
(a) Real 2D Scatter-plot



(b) Final 2D Scatter-plot

The figure 10a is a 2D scatter-plot of the first two principal components (PCs) of the iris data matrix  $X$ , grouped by the real partitioning into cluster, according to the  $I$  column vector, contained into the Matlab data file IrisDataAnnotated.mat.

Figure 10: a) 2D scatter-plot of the original matrix according to the  $I$  column vector. b-c) The 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the final clustering results.



(a) Final 3D Scatter-plot

4. If our criterion, in this case, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , is lower the tolerance,  $\tau$ , fixed before, the algorithm can stop; otherwise, the counter continues (it is set  $t = t + 1$ ), and the algorithm goes to the step 3.

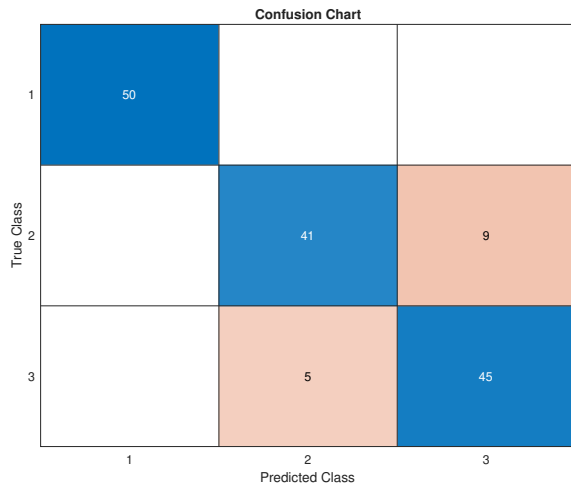
**If**

$$|Q(\Pi_t) - Q(\Pi_{t+1})| < \tau \quad \text{stop}$$

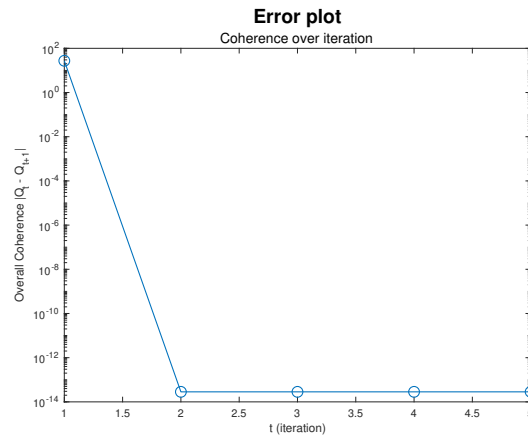
**else**

$$t = t + 1, \quad \text{go to step 3.}$$

The progress of the criterion, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , could be seen at figure 11c. It can be notice that after 2 iteration, the criterion reach 0, meaning that the overall coherence of the partitioning are equal  $Q(\Pi_t) = Q(\Pi_{t+1})$



(b) Confusion Chart



(c) Overall Coherence plot

Figure 11: a) Confusion Chart derived from the comparison of the  $I\_assign$  (produced by my *k-medoids* clustering) and the  $I$  column vectors. b) Plot of the difference between the overall coherence of preceding clustering,  $Q_t$ , and of the last one,  $Q_{t+1}$ ,  $|Q_t - Q_{t+1}|$ , over the iteration,  $t$ .

The figure 11b is a visualisation of the confusion matrix (indeed, the confusion chart) in which can be seen the errors that my algorithm are made, comparing the  $I$  and my  $I\_assign$  column vector: In this case,

- the *k-means* found all the points associated to the first cluster, 50/50, with an error of 0%.
- for the second cluster, the algorithm found 41/50, 41 points belong to the second cluster out of the total 50, incorrectly assigning the last 9 to third cluster, producing an error of 18%.
- the clustering technique found 45 points belonging to the third cluster, erroneously assigning 5 vectors tot the second cluster, emitting an error of 10%.



### CardiacSPECT Dataset

The data file **CardiacSPECT.mat**, originating from the UCI Machine Learning Repository site, contains a binary matrix  $X$  of size  $22 \times 187$ ,  $X \in \mathbb{R}^{22 \times 187}$ , and a vector  $I$  of length 187,  $I \in \mathbb{R}^{1 \times 187}$ . The dataset reports 22 attributes extracted from cardiac Single Proton Emission Computed Tomography (SPECT) images of 187 patients. Each patient was classified into one of two categories: normal or abnormal. The database of 187 SPECT image sets (patients) was processed to extract features that summarize the original SPECT images. As a result, 44 continuous feature pattern was created for each patient. The pattern was further processed to obtain 22 binary feature patterns:

$$x^j = \begin{bmatrix} F1 \\ F2 \\ F3 \\ F4 \\ F5 \\ F6 \\ F7 \\ F8 \\ F9 \\ F10 \\ F11 \\ F12 \\ F13 \\ F14 \\ F15 \\ F16 \\ F17 \\ F18 \\ F19 \\ F20 \\ F21 \\ F22 \end{bmatrix} \in \mathbb{R}^{22}, \quad 1 \leq j \leq 187$$

The vector  $I$ , also in binary form, contains the annotation of the corresponding patient in one of the groups.

Before diving into the clustering techniques, my approach is to compute a dimensional reduction, using Principal Component Analysis (PCA), on centering data matrix,  $X_c = X - \bar{X}$ , for the only goal of better visualizing the result since we have already studied that the first Principal Component (PCs) (in the wine data-set, 2 – 3, depending on the choice of a two- or three-dimensional visualisation) are the ones that maximize the spread of the data. So, we compute the SVD of the centering data points,  $X_c = UDV'$ , for then computing the Principal Components (PCs),  $Z$ .

Our aim is to apply my *k-medoids* (PAM (Partitioning Around Medoids)) sub-routine function to the CardiacSPECT Matrix,  $X$ :

1. First of all, fix  $k$ , the number of clusters (in this case,  $k = 2$ ) and a positive tolerance,  $\tau > 0$  (for this example,  $\tau = 1.0e^{-100}$ ), used as criterion to stop the for-loop iteration;

2. Then, the **initialisation step** in which it is chosen the initial set of medoids and the counting is started (it is set  $t = 0$ ): In this case, the initial characteristic vectors are picked randomly (with the difference that, with the respect of *k-means*, they belong to one of the data point of the cluster,  $c_0^{(l)} \in D_l$ ) ( $\{c_0^{(1)}, c_0^{(2)} | c_0^l \in \mathbb{R}^{22} | 1 \leq l \leq 2\}$ ) (see Table 7). This part can be clearly seen in figure 12a, for a 2D representation, and in figure 12b, for a three-dimensional one;

| CardiacSPECT |         |         |
|--------------|---------|---------|
|              | Group A | Group B |
| F1           | 1       | 1       |
| F2           | 1       | 0       |
| F3           | 0       | 0       |
| F4           | 0       | 0       |
| F5           | 1       | 0       |
| F6           | 1       | 0       |
| F7           | 0       | 0       |
| F8           | 0       | 0       |
| F9           | 0       | 1       |
| F10          | 1       | 0       |
| F11          | 1       | 0       |
| F12          | 0       | 0       |
| F13          | 0       | 0       |
| F14          | 0       | 1       |
| F15          | 1       | 0       |
| F16          | 1       | 0       |
| F17          | 1       | 0       |
| F18          | 0       | 0       |
| F19          | 0       | 0       |
| F20          | 1       | 0       |
| F21          | 1       | 0       |
| F22          | 0       | 1       |

Table 7: Initial medoids

CardiacSPECT Dataset

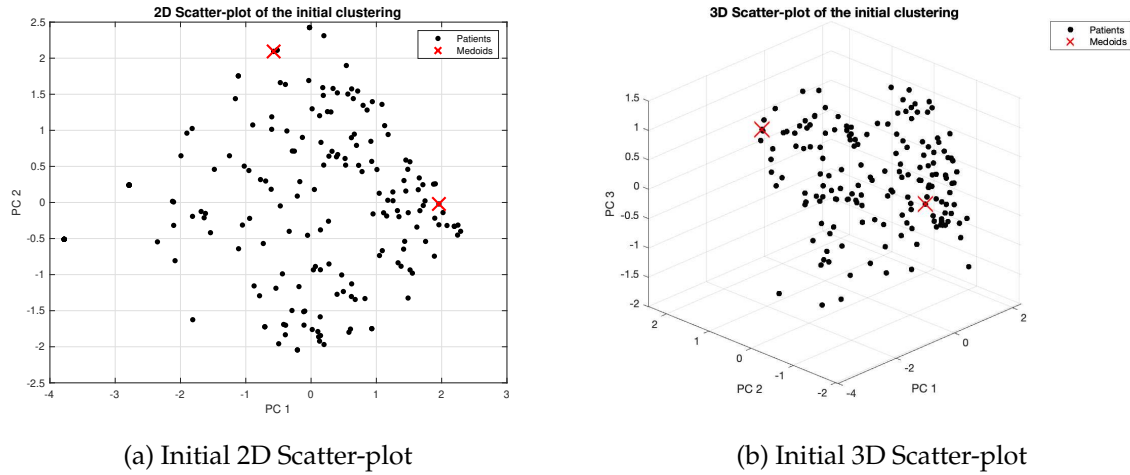


Figure 12: a-b) 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the initial seed of the characteristic vectors, the medoids,  $c^{(l)}$ , randomly chosen.

### 3. Iteration step:

- (a) **Assignment step:**  $\forall x^j, \quad 1 \leq j \leq 150$ , it is computed the minimum distance between each data points and the initial medoids, randomly chosen. In this case, since data are in binary format (made of 0 and 1's), the norm is not a suitable measure. The most appropriate distance measure is the **Jaccard dissimilarity index**, defined in terms of the Jaccard similarity index (it is, indeed  $1 - \text{the Jaccard similarity index}$ ). For two given sets, the Jaccard similarity index is defined as the ratio of the intersection of the two sets divided by their union: for binary values, the definition is the following:

$$d(x^j, c_t^{(l)}) = \min_{1 \leq l \leq k} d(x^j, c_t^{(l)})$$

$$d(x^j, c_t^{(l)}) = 1 - \frac{n_{11}}{n_{10} + n_{01} + n_{11}} = \frac{n_{10} + n_{01}}{n_{10} + n_{01} + n_{11}}$$

where:

- $n_{10}$ : the number of couple of data points data that takes on value 1 for a column and 0 for the other;
- $n_{01}$ : the number of couple of data points that takes on value 0 for a column and 1 for the other;
- $n_{11}$ : the number of matching ones: couple of data points that takes on value 1 for a column and 1 for the other.

and assign each data points,  $x^j$ , to the corresponding (nearest) cluster. This process defines a partition,  $\Pi_t$ , for which it is possible to compute the within-cluster tightness,  $q_l(c_t^{(l)})$ , and the overall coherence,  $Q(\Pi_t)$ .

- (b) **Updating step**, in which for each cluster,  $\forall D_l$ , it is made a local distance matrix consisting of the distance between the data points of the corresponding cluster. So, it is used this matrix to interchange the medoids,  $c_t^{(l)}$ , with each

data points of that cluster,  $x^j$ , in order to find the point that minimizes the within-cluster coherence: that it is an easy task since we need to find the minimum sum of the points of each column, the minimum one is the chosen. Then, the within-cluster tightness is recomputed using as medoids the data point chosen,  $q_l(x^j)$ .

If the within-cluster coherence of the data point,  $q_l(x^j)$ , is minor than the within-cluster tightness of the medoid,  $c_t^{(l)}$ , then the data point is interchange with the medoid and this applied also with the within-cluster coherence.

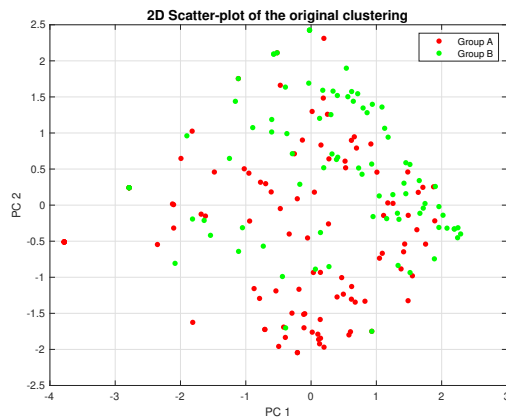
**If**

$$q_l(x^j) < q_l(c_t^{(l)})$$

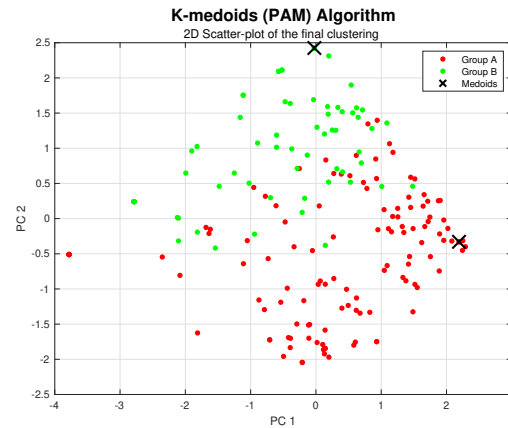
**then**

$$c_t^{(l)} = x^j, \quad q_l(c_t^{(l)}) = q_l(x^j)$$

This process produces the final medoids, that can be seen, numerically, in the table 8 and, visually, on the figure 13b and 14a, where it is also possible to see the final clustering of the data into three cluster in a bi-dimensional view i the first figure and in a three- one in the second one.



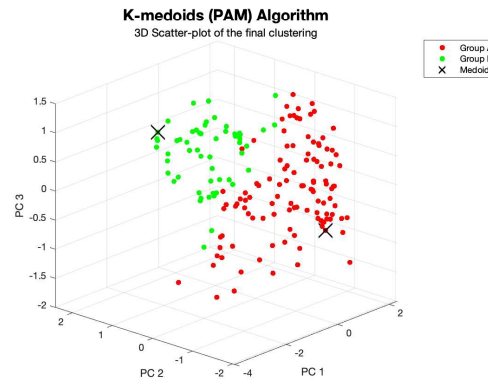
(a) Real 2D Scatter-plot



(b) Final 2D Scatter-plot

The figure 13a is a 2D scatter-plot of the first two principal components (PCs) of the CardiacSPECT data matrix  $X$ , grouped by the real partitioning into cluster, according to the  $I$  column vector, contained into the Matlab data file CardiacSPECT.mat.

Figure 13: a) 2D scatter-plot of the original matrix according to the  $I$  column vector. b-c) The 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the final clustering results.



(a) Final 3D Scatter-plot

| CardiacSPECT |         |         |
|--------------|---------|---------|
|              | Group A | Group B |
| F1           | 1       | 1       |
| F2           | 1       | 0       |
| F3           | 0       | 0       |
| F4           | 1       | 1       |
| F5           | 1       | 0       |
| F6           | 1       | 0       |
| F7           | 0       | 0       |
| F8           | 0       | 0       |
| F9           | 1       | 1       |
| F10          | 1       | 0       |
| F11          | 1       | 0       |
| F12          | 0       | 0       |
| F13          | 0       | 0       |
| F14          | 1       | 1       |
| F15          | 1       | 0       |
| F16          | 1       | 0       |
| F17          | 1       | 0       |
| F18          | 1       | 0       |
| F19          | 0       | 0       |
| F20          | 1       | 0       |
| F21          | 1       | 0       |
| F22          | 1       | 1       |

Table 8: Final medoids

## CardiacSPECT Dataset

4. If our criterion, in this case, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , is lower the tolerance,  $\tau$ , fixed before, the algorithm can stop; otherwise, the counter continues (it is set  $t = t + 1$ ), and the algorithm goes to the step 3.

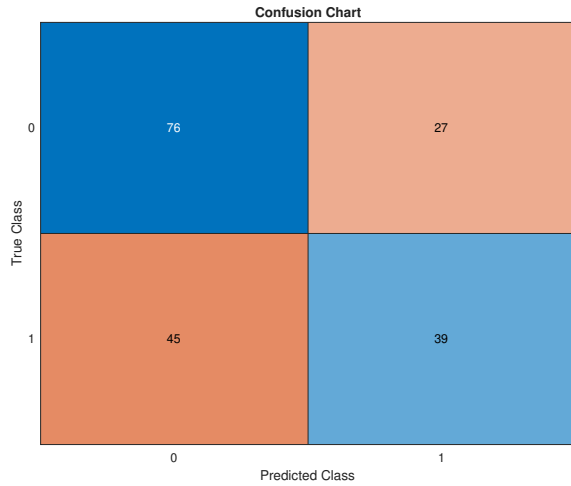
**If**

$$|Q(\Pi_t) - Q(\Pi_{t+1})| < \tau \quad \textbf{stop}$$

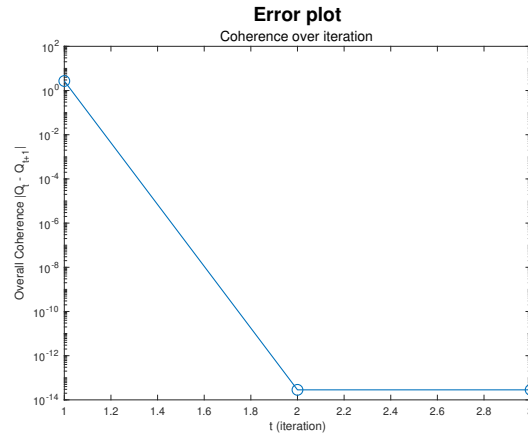
**else**

$$t = t + 1, \quad \text{go to step 3.}$$

The progress of the criterion, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , could be seen at figure 14c. It can be notice that after 2 iteration, the criterion reach 0, meaning that the overall coherence of the partitioning are equal  $Q(\Pi_t) = Q(\Pi_{t+1})$



(b) Confusion Chart



(c) Overall Coherence plot

Figure 14: a) Confusion Chart derived from the comparison of the  $I_{assign}$  (produced by my *k-medoids* clustering) and the  $I$  column vectors. b) Plot of the difference between the overall coherence of preceding clustering,  $Q_t$ , and of the last one,  $Q_{t+1}$ ,  $|Q_t - Q_{t+1}|$ , over the iteration,  $t$ .

The figure 14b is a visualisation of the confusion matrix (indeed, the confusion chart) in which can be seen the errors that my algorithm are made, comparing the  $I$  and my  $I_{assign}$  column vector: In this case,

- the *k-means* found 76 points associated to the first cluster out of the total 103, 76/103, Mistakenly assigning 27 points to the second cluster, with an error of 26.21%.
- for the second cluster, the algorithm found 39/84, 34 points belong to the second cluster out of the total 84, incorrectly assigning the last 45 to first cluster, producing an error of 53.57%.

### Wine Dataset

the data file **WineData.mat** containing a data matrix  $X$ ,  $X \in \mathbb{R}^{13 \times 178}$ , of chemical analysis data of wines derived from the same area of Italy but originating from three different cultivars. The original data file can be found on the UCI Machine Learning Repository. The attributes, corresponding to the rows of the data matrix  $X$  are concentrations/levels of the following substances:

$$x^j = \begin{bmatrix} \text{Alcohol} \\ \text{Malic acid} \\ \text{Ash} \\ \text{Alcalinity of ash} \\ \text{Magnesium} \\ \text{Total phenols} \\ \text{Flavonoids} \\ \text{Nonflavonoid phenols} \\ \text{Proanthocyanins} \\ \text{Color intensity} \\ \text{Hue} \\ \text{OD280/OD315 of diluted wines} \\ \text{Proline} \end{bmatrix} \in \mathbb{R}^{13}, \quad 1 \leq j \leq 178$$

**K-means.** Before diving into the clustering techniques, my approach is to compute a dimensional reduction, using Principal Component Analysis (PCA), on centering data matrix,  $X_c = X - \bar{X}$ , for the only goal of better visualizing the result since we have already studied that the first Principal Component (PCs) (in the Wine data-set, 2 – 3, depending on the choice of a two- or three-dimensional visualisation) are the ones that maximize the spread of the data. So, we compute the SVD of the centering data points,  $X_c = UDV'$ , for then computing the Principal Components (PCs),  $Z$ .

Our aim is to apply my *k-means* (Lloyd's) sub-routine function to the Iris Matrix,  $X$ :

1. First of all, fix  $k$ , the number of clusters (in this case,  $k = 3$ ) and a positive tolerance,  $\tau > 0$  (for this example,  $\tau = 1.0e^{-100}$ ), used as criterion to stop the for-loop iteration;
2. Then, the **initialisation step** in which it is defined a starting partition,  $\Pi_0$ , and the counting is started (it is set  $t = 0$ ): In this case, the initial initialisation is to pick random seed vectors ( $\{c_0^{(1)}, c_0^{(2)}, c_0^{(3)} | c_0^l \in \mathbb{R}^{13} | 1 \leq l \leq 3\}$ ) (see Table 9) and divide the data into cluster of , approximately, the same sizes. This part can be clearly seen in figure 15a, for a 2D representation, and in figure 15b, for a three-dimensional one;

| Italian Wine                        |            |            |            |
|-------------------------------------|------------|------------|------------|
|                                     | Cultivar 1 | Cultivar 2 | Cultivar 3 |
| Alcohol                             | -3.77      | 237.24     | 97.13      |
| Malic acid                          | -36.03     | 17.53      | 13.06      |
| Ash                                 | 6.13       | 0.66       | 1.68       |
| Alcalinity of ash                   | -0.37      | 0.10       | 1.64       |
| Magnesium                           | -0.28      | 1.28       | -0.27      |
| Total phenols                       | 1.11       | 0.63       | 1.83       |
| Flavonoids                          | -0.71      | 0.10       | -0.27      |
| Nonflavonoid phenols of ash         | -1.08      | 0.53       | -0.26      |
| Proanthocyanins                     | -0.01      | 0.80       | -0.28      |
| Color intensity acid                | 0.16       | 0.08       | -0.51      |
| Hue                                 | -0.02      | -0.32      | 0.23       |
| OD280/OD315 of diluted wines of ash | -0.03      | 0.15       | -0.16      |
| Proline of ash                      | 0.10       | -0.04      | 0.11       |

Table 9: Initial seed vector

Iris Dataset

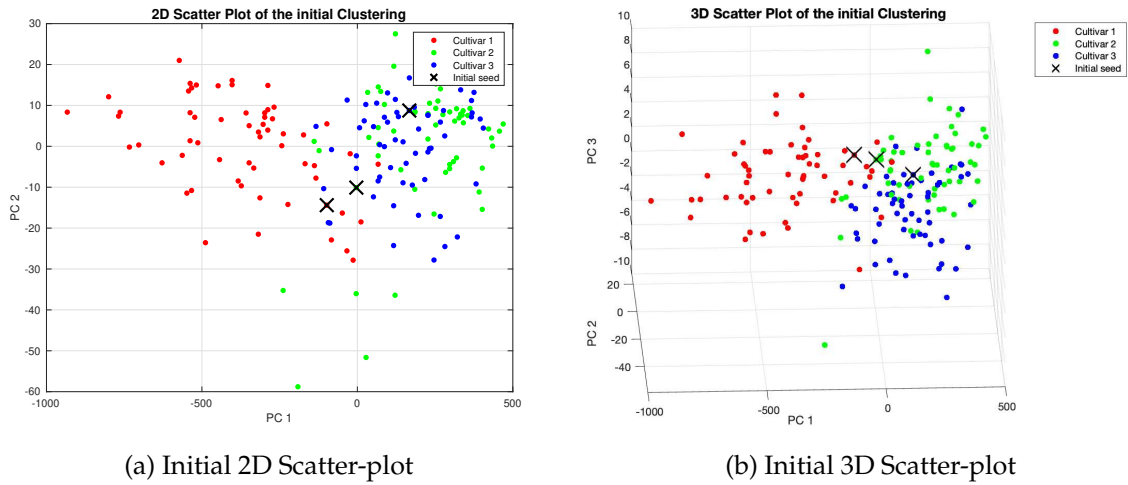


Figure 15: a-b) 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the initial seed vectors, randomly chosen.

### 3. Iteration step:

(a) **Updating step**, in which it is computed the centroid,  $c_t^{(l)}$ , of the partition,  $\Pi_t$ :

$$c_t^{(l)} = \frac{1}{|D_l|} \sum_{j \in I_l} x^j$$

(b) **Assignment step**:  $\forall x^j, \quad 1 \leq j \leq 178$ , it is computed the minimum distance from the centroids,  $c_t^{(l)}$ , using the euclidean norm (the final centroids, result



of my algorithm, can be found at table 11)

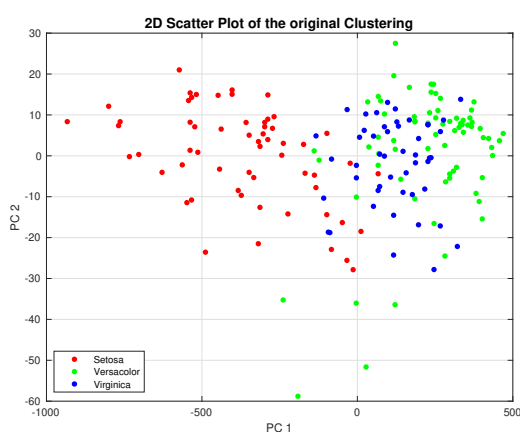
$$d(x^j - c_t^{(l)}) = \|x^j - c_t^{(l)}\|_2$$

and assign each data points,  $x^j$ , to the corresponding (nearest) cluster. This process defines a new partition,  $\Pi_{t+1}$ . (The new partition, as the consequent final clustering of the Wine data) could be seen at figure 16b, for a bi-dimensional visualisation, and at figure 17a, for a 3D one).

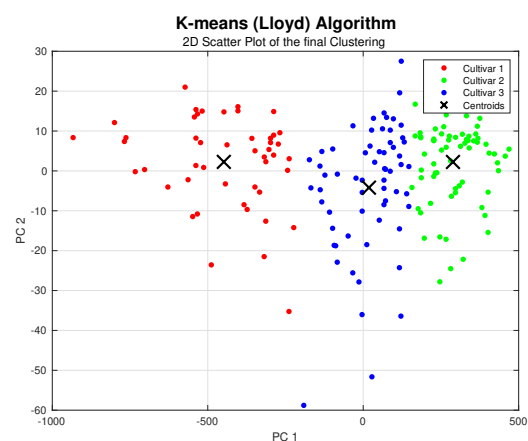
| Italian Wine                        |            |            |            |
|-------------------------------------|------------|------------|------------|
|                                     | Cultivar 1 | Cultivar 2 | Cultivar 3 |
| Alcohol                             | -448.30    | 288.75     | 18.48      |
| Malic acid                          | 2.26       | 2.24       | -4.20      |
| Ash                                 | 0.47       | 0.07       | -0.43      |
| Alcalinity of ash                   | -0.32      | -0.30      | 0.59       |
| Magnesium                           | 0.20       | 0.08       | -0.24      |
| Total phenols                       | -0.14      | -0.10      | 0.22       |
| Flavonoids                          | 0.10       | 0.02       | -0.10      |
| Nonflavonoid phenols of ash         | 0.02       | 0.01       | -0.02      |
| Proanthocyanins                     | -0.02      | -0.04      | 0.06       |
| Color intensity acid                | 0.02       | 0.03       | -0.05      |
| Hue                                 | 0.00       | -0.02      | 0.04       |
| OD280/OD315 of diluted wines of ash | 0.00       | 0.01       | -0.01      |
| Proline of ash                      | 0.00       | 0.00       | -0.00      |

Table 10: Final centroid vector

Iris Dataset



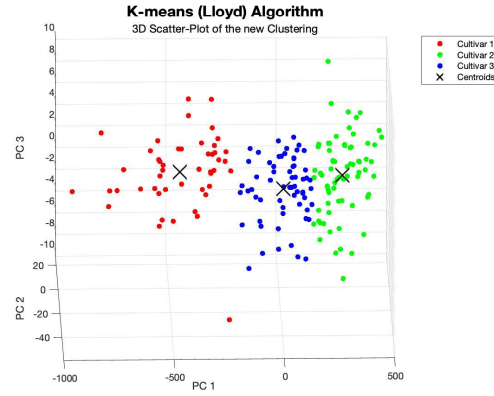
(a) Real 2D Scatter-plot



(b) Final 2D Scatter-plot

The figure 16a is a 2D scatter-plot of the first two principal components (PCs) of the Wine data matrix  $X$ , grouped by the real partitioning into cluster, according to the  $I$  column vector, contained into the Matlab data file WineData.mat.

Figure 16: a) 2D scatter-plot of the original matrix according to the  $I$  column vector. b-c) The 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the final clustering results.



(a) Final 3D Scatter-plot

4. If our criterion, in this case, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , is lower the tolerance,  $\tau$ , fixed before, the algorithm can stop; otherwise, the counter continues (it is set  $t = t + 1$ ), and the algorithm goes to the step 3.

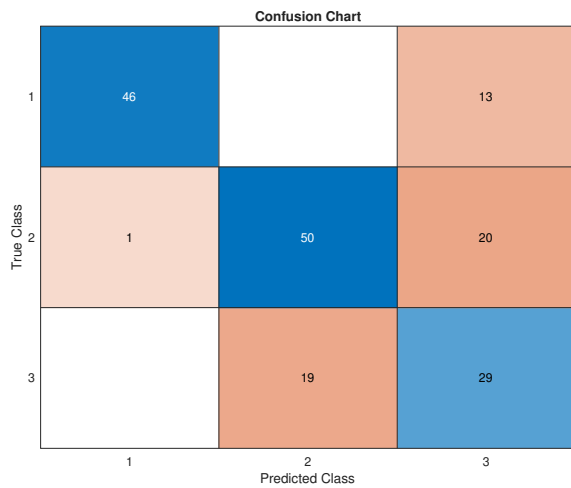
**If**

$$|Q(\Pi_t) - Q(\Pi_{t+1})| < \tau \quad \text{stop}$$

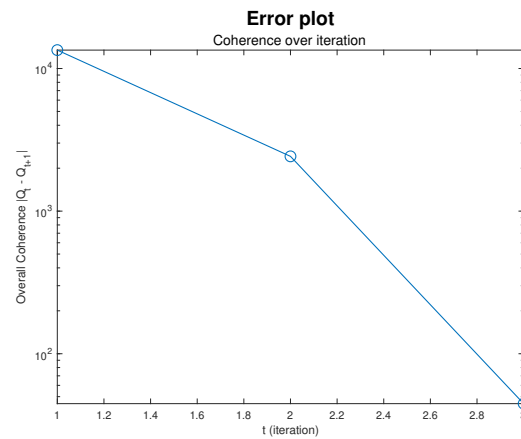
**else**

$$t = t + 1, \quad \text{go to step 3.}$$

The progress of the criterion, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , could be seen at figure 17c. It can be notice that after 3 iteration, the criterion reach 0, meaning that the overall coherence of the partitioning are equal  $Q(\Pi_t) = Q(\Pi_{t+1})$



(b) Confusion Chart



(c) Overall Coherence plot

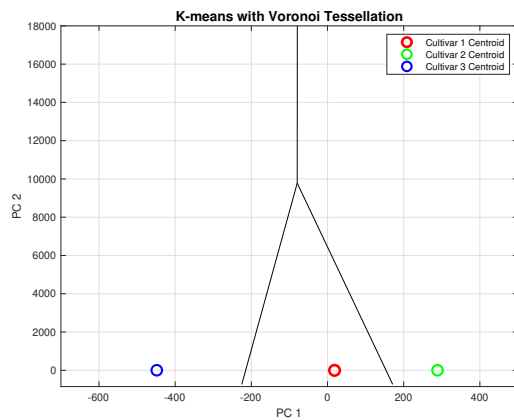
Figure 17: a) Confusion Chart derived from the comparison of the  $I_{assign}$  (produced by my  $k$ -means clustering) and the  $I$  column vectors. b) Plot of the difference between the overall coherence of preceding clustering,  $Q_t$ , and of the last one,  $Q_{t+1}$ ,  $|Q_t - Q_{t+1}|$ , over the iteration,  $t$ .

The figure 17b is a visualisation of the confusion matrix (indeed, the confusion chart) in which can be seen the errors that my algorithm are made, comparing the  $I$  and my  $I\_assign$  column vector: In this case,

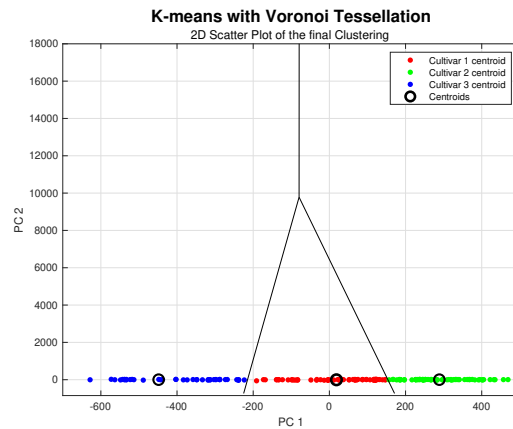
- the  $k$ -means found 46 points associated to the first cluster out of the total 59, 46/59, Mistakenly assigning 13 points to the third cluster, with an error of 22.03%.
- for the second cluster, the algorithm found 50/71, 50 points belong to the second cluster out of the total 71, incorrectly assigning the last 20 to third cluster and 1 to the first cluster, producing an error of 29.58%.
- the clustering technique found 29 points belonging to the third cluster out of 48 total vectors, erroneously assigning 19 vectors tot the second cluster, emitting an error of 39.58%.

The assignment step of the  $k$ -means (Lloyd's) Algorithm has a geometrical meaning: it produces, from a geometric view point, a **Voronoi tessellation**. A Voronoi tessellation is a space of  $\mathbb{R}^{n-1}$  in which each cluster is separated by hyper-planes such that each hyper-plane is equidistant from two centroids. The sets is of size  $n - 1$  because it creates  $n - 1$  clusters divided by hyper-plane. A proof of this theoretical foundation is the Voronoi tessellation (see figure 18a), produced by my  $k$ -means algorithm (on Matlab) with a pre-fixed number of cluster  $k = 3$ , and the same with also the 2D scatter-plot of the principal components (PCs) of the data, grouped by the predicted classes (see figure 18b).

For a better visualisation of geometrical implication of the assignment step, the Voronoi tessellation, the figure 19a and the consequent one 19b are representation of the Voronoi sets when the pre-fixed number of cluster  $k = 10$ .



(a) Voronoi tessellation



(b) 2D Scatter-plot with Voronoi sets

Figure 18: a) Voronoi tessellation of the final centroids obtained from the  $k$ -means clustering technique with fixed  $k = 3$  cluster. b) 2D scatter-plot of the final  $k$ -means clustering of the data with Voronoi sets with fixed  $k = 3$  cluster.

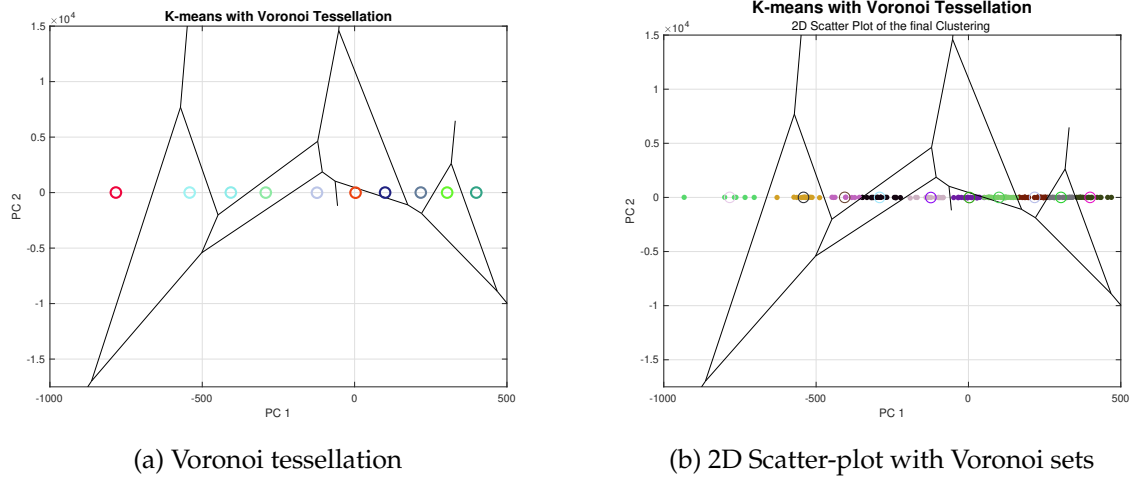


Figure 19: a) Voronoi tessellation of the final centroids obtained from the *k-means* clustering technique with fixed  $k = 10$  cluster. b) 2D scatter-plot of the final *k-means* clustering of the data with fixed  $k = 10$  cluster with Voronoi sets.

**K-medoids.** Before diving into the clustering techniques, my approach is to compute a dimensional reduction, using Principal Component Analysis (PCA), on centering data matrix,  $X_c = X - \bar{X}$ , for the only goal of better visualizing the result since we have already studied that the first Principal Component (PCs) (in the wine data-set, 2 – 3, depending on the choice of a two- or three-dimensional visualisation) are the ones that maximize the spread of the data. So, we compute the SVD of the centering data points,  $X_c = UDV'$ , for then computing the Principal Components (PCs),  $Z$ .

Our aim is to apply my *k-medoids* (PAM (Partitioning Around Medoids)) sub-routine function to the Wine Matrix,  $X$ :

1. First of all, fix  $k$ , the number of clusters (in this case,  $k = 3$ ) and a positive tolerance,  $\tau > 0$  (for this example,  $\tau = 1.0e^{-100}$ ), used as criterion to stop the for-loop iteration;
2. Then, the **initialisation step** in which it is chosen the initial set of medoids and the counting is started (it is set  $t = 0$ ): In this case, the initial characteristic vectors are picked randomly (with the difference that, with the respect of *k-means*, they belong to one of the data point of the cluster,  $c_0^{(l)} \in D_l$ ) ( $\{c_0^{(1)}, c_0^{(2)}, c_0^{(3)} | c_0^{(l)} \in \mathbb{R}^4 | 1 \leq l \leq 3\}$ ) (see Table 11). This part can be clearly seen in figure 20a, for a 2D representation, and in figure 20b, for a three-dimensional one;

| Italian Wine                        |            |            |            |
|-------------------------------------|------------|------------|------------|
|                                     | Cultivar 1 | Cultivar 2 | Cultivar 3 |
| Alcohol                             | -488.60    | 296.82     | 27.97      |
| Malic acid                          | -23.57     | -5.47      | -51.63     |
| Ash                                 | -1.17      | 3.66       | 5.00       |
| Alcalinity of ash                   | -1.66      | 2.58       | -1.51      |
| Magnesium                           | -0.06      | 0.40       | -0.64      |
| Total phenols                       | 0.56       | 1.26       | 1.13       |
| Flavonoids                          | 0.41       | 0.18       | -0.48      |
| Nonflavonoid phenols of ash         | 0.49       | 0.54       | -0.87      |
| Proanthocyanins                     | -0.07      | -0.52      | 0.86       |
| Color intensity acid                | -0.18      | -0.08      | -0.12      |
| Hue                                 | 0.05       | -0.14      | -0.31      |
| OD280/OD315 of diluted wines of ash | 0.17       | 0.11       | 0.23       |
| Proline of ash                      | 0.19       | 0.14       | -0.07      |

Table 11: Initial medoids

## Wine Dataset

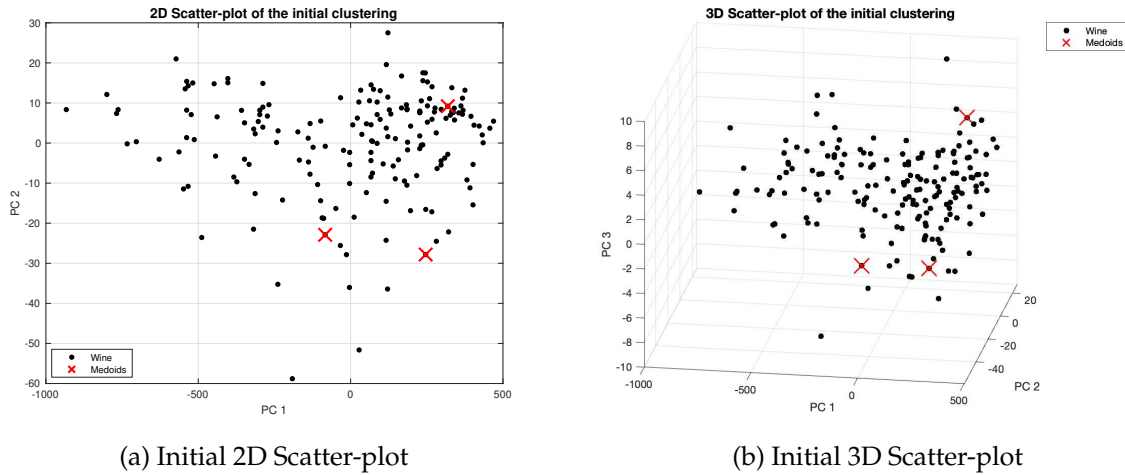


Figure 20: a-b) 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the initial seed of the characteristic vectors, the medoids,  $c^{(l)}$ , randomly chosen.

### 3. Iteration step:

- (a) **Assignment step:**  $\forall x^j, \quad 1 \leq j \leq 150$ , it is computed the minimum distance between each data points and the initial medoids, randomly chosen, (in this case, the distance measure used is the  $2norm$ )

$$d(x^j, c_t^{(l)}) = \min_{1 \leq l \leq k} d(x^j, c_t^{(l)})$$

$$d(x^j, c_t^{(l)}) = \|x^j - c_t^{(l)}\|_2$$

and assign each data points,  $x^j$ , to the corresponding (nearest) cluster. This process defines a partition,  $\Pi_t$ , for which it is possible to compute the within-cluster tightness,  $q_l(c_t^{(l)})$ , and the overall coherence,  $Q(\Pi_t)$ .

- (b) **Updating step**, in which for each cluster,  $\forall D_l$ , it is made a local distance matrix consisting of the distance between the data points of the corresponding cluster. So, it is used this matrix to interchange the medoids,  $c_t^{(l)}$ , with each data points of that cluster,  $x^j$ , in order to find the point that minimizes the within-cluster coherence: that it is an easy task since we need to find the minimum sum of the points of each column, the minimum one is the chosen. Then, the within-cluster tightness is recomputed using as medoids the data point chosen,  $q_l(x^j)$ .

If the within-cluster coherence of the data point,  $q_l(x^j)$ , is minor than the within-cluster tightness of the medoid,  $c_t^{(l)}$ , then the data point is interchange with the medoid and this applied also with the within-cluster coherence.

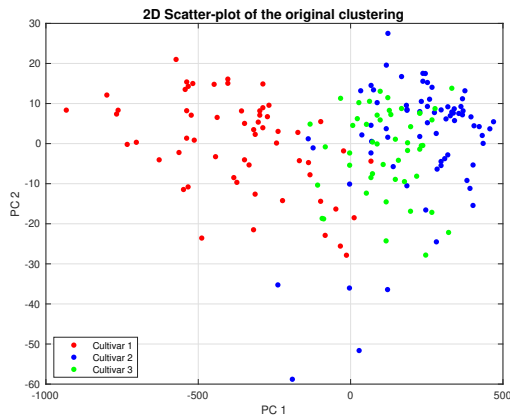
**If**

$$q_l(x^j) < q_l(c_t^{(l)})$$

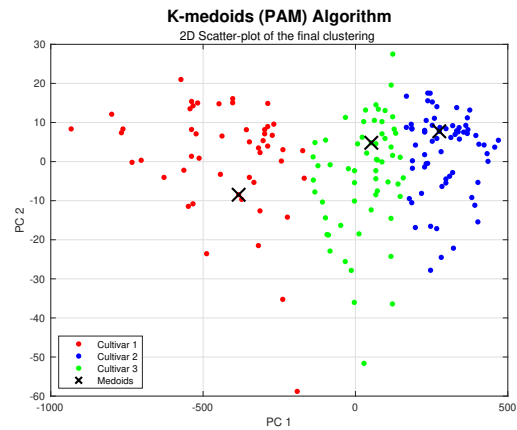
**then**

$$c_t^{(l)} = x^j, \quad q_l(c_t^{(l)}) = q_l(x^j)$$

This process produces the final medoids, that can be seen, numerically, in the table 12 and, visually, on the figure 21b and 22a, where it is also possible to see the final clustering of the data into three cluster in a bi-dimensional view i the first figure and in a three- one in the second one.



(a) Real 2D Scatter-plot



(b) Final 2D Scatter-plot

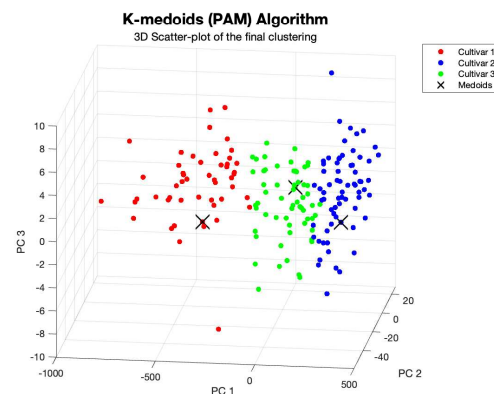
| Italian Wine                        |            |            |            |
|-------------------------------------|------------|------------|------------|
|                                     | Cultivar 1 | Cultivar 2 | Cultivar 3 |
| Alcohol                             | -438.06    | 275.10     | 36.93      |
| Malic acid                          | 6.54       | 7.75       | 2.17       |
| Ash                                 | -1.11      | -2.93      | 2.59       |
| Alcalinity of ash                   | -0.91      | -1.70      | -1.01      |
| Magnesium                           | 0.38       | 0.66       | -0.37      |
| Total phenols                       | -0.67      | 0.45       | 0.21       |
| Flavonoids                          | -0.79      | 1.32       | -0.57      |
| Nonflavonoid phenols of ash         | -0.50      | -0.02      | 0.28       |
| Proanthocyanins                     | 0.02       | 0.09       | 0.21       |
| Color intensity acid                | 0.21       | 0.20       | 0.04       |
| Hue                                 | 0.24       | -0.30      | -0.50      |
| OD280/OD315 of diluted wines of ash | -0.05      | -0.04      | 0.28       |
| Proline of ash                      | -0.04      | -0.07      | -0.01      |

Table 12: Final medoids

## Wine Dataset

The figure 21a is a 2D scatter-plot of the first two principal components (PCs) of the wine data matrix  $X$ , grouped by the real partitioning into cluster, according to the  $I$  column vector, contained into the Matlab data file WineData.mat.

Figure 21: a) 2D scatter-plot of the original matrix according to the  $I$  column vector. b-c) The 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the final clustering results.



(a) Final 3D Scatter-plot

- If our criterion, in this case, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , is lower the tolerance,  $\tau$ , fixed before, the algorithm can stop; otherwise, the counter continues (it is set  $t = t + 1$ ), and the algorithm goes to the step 3.

**If**

$$|Q(\Pi_t) - Q(\Pi_{t+1})| < \tau \quad \text{stop}$$

**else**

$$t = t + 1, \quad \text{go to step 3.}$$

The progress of the criterion, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , could be seen at

figure 22c. It can be notice that after 7 iteration, the criterion reach 0, meaning that the overall coherence of the partitioning are equal  $Q(\Pi_t) = Q(\Pi_{t+1})$

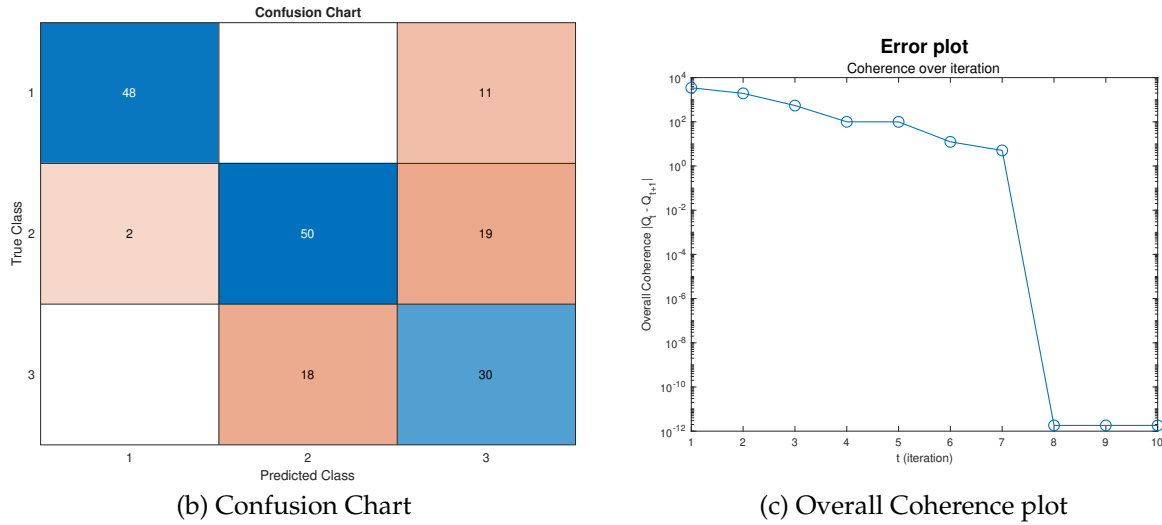


Figure 22: a) Confusion Chart derived from the comparison of the  $I_{assign}$  (produced by my *k-medoids* clustering) and the *I* column vectors. b) Plot of the difference between the overall coherence of the current iteration and of the last one,  $Q_t - Q_{t+1}$ , over the iteration,  $t$ .

The figure 22b is a visualisation of the confusion matrix (indeed, the confusion chart) in which can be seen the errors that my algorithm are made, comparing the *I* and my *I\_assign* column vector: In this case,

- the *k-means* found 48 points associated to the first cluster out of the total 59, 48/59, Mistakenly assigning 11 points to the third cluster, with an error of 18.65%.
- for the second cluster, the algorithm found 50/71, 50 points belong to the second cluster out of the total 71, incorrectly assigning the last 19 to third cluster and 2 to the first cluster, producing an error of 29.58%.
- the clustering technique found 30 points belonging to the third cluster out of 48 total vectors, erroneously assigning 18 vectors tot the second cluster, emitting an error of 37.5%.



### Congressional Vote Dataset

The Matlab data file **CongressionalVoteData.mat**, originating also from the UCI Machine Learning Repository, contains a data matrix  $X$ ,  $X \in \mathbb{R}^{16 \times 435}$ , containing the votes of the 435 congressional representatives in 1984 on 16 issues, as well as a vector  $I$  of length 435,  $I \in \mathbb{R}^{1 \times 435}$ , indicating their partisan membership of each representative (**Republican** = 0, **Democrat** = 1). The columns of the Matrix  $X$  give a yes/no vote of each congressional representative on the following 16 issues (“yes” = 1, “no” = 0, “missing vote” = 0):

$$x^j = \begin{bmatrix} \text{Handicapped infants} \\ \text{Water project cost sharing} \\ \text{Adoption of the budget resolution} \\ \text{Physician fee freeze} \\ \text{El Salvador aid} \\ \text{Religious groups in schools} \\ \text{Anti-satellite test ban} \\ \text{Aid to Nicaraguan contras} \\ \text{Mx missile} \\ \text{Immigration} \\ \text{Synfuels corporation cutback} \\ \text{Education spending} \\ \text{Superfund right to sue} \\ \text{Crime} \\ \text{Duty free exports} \\ \text{Export administration act South Africa} \end{bmatrix} \in \mathbb{R}^{16}, \quad 1 \leq j \leq 435$$

Before diving into the clustering techniques, my approach is to compute a dimensional reduction, using Principal Component Analysis (PCA), on centering data matrix,  $X_c = X - \bar{X}$ , for the only goal of better visualizing the result since we have already studied that the first Principal Component (PCs) (in the wine data-set, 2 – 3, depending on the choice of a two- or three-dimensional visualisation) are the ones that maximize the spread of the data. So, we compute the SVD of the centering data points,  $X_c = UDV'$ , for then computing the Principal Components (PCs),  $Z$ .

Our aim is to apply my *k-medoids* (PAM (Partitioning Around Medoids)) sub-routine function to the Congressional Vote Matrix,  $X$ :

1. First of all, fix  $k$ , the number of clusters (in this case,  $k = 2$ ) and a positive tolerance,  $\tau > 0$  (for this example,  $\tau = 1.0e^{-100}$ ), used as criterion to stop the for-loop iteration;
2. Then, the **initialisation step** in which it is chosen the initial set of medoids and the counting is started (it is set  $t = 0$ ): In this case, the initial characteristic vectors are picked randomly (with the difference that, with the respect of *k-means*, they belong to one of the data point of the cluster,  $c_0^{(l)} \in D_l$ ) ( $\{c_0^{(1)}, c_0^{(2)} | c_0^{(l)} \in \mathbb{R}^{16} | 1 \leq l \leq 2\}$ ) (see Table 13). This part can be clearly seen in figure 23a, for a 2D representation, and in figure 23b, for a three-dimensional one;

| Congressional Vote                     |            |          |
|----------------------------------------|------------|----------|
|                                        | Republican | Democrat |
| Handicapped infants                    | 1          | 1        |
| Water project cost sharing             | -1         | 1        |
| Adoption of the budget resolution      | 1          | 1        |
| Physician fee freeze                   | -1         | -1       |
| El Salvador aid                        | -1         | -1       |
| Religious groups in schools            | -1         | -1       |
| Anti-satellite test ban                | 1          | 1        |
| Aid to Nicaraguan contras              | 1          | 1        |
| Mx missile                             | 1          | 1        |
| Immigration                            | 1          | -1       |
| Synfuels corporation cutback           | 1          | 1        |
| Education spending                     | -1         | -1       |
| Superfund right to sue                 | 1          | -1       |
| Crime                                  | -1         | -1       |
| Duty free exports                      | 1          | -1       |
| Export administration act South Africa | 1          | 1        |

Table 13: Initial medoids

## Congressional Vote Dataset

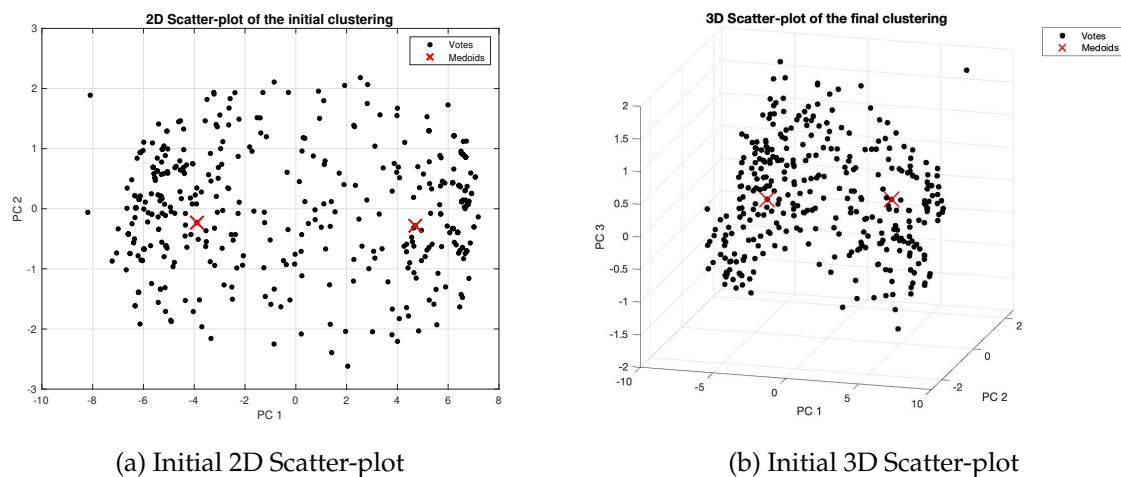


Figure 23: a-b) 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the initial seed of the characteristic vectors, the medoids,  $c^{(l)}$ , randomly chosen.

### 3. Iteration step:

- (a) **Assignment step:**  $\forall x^j, \quad 1 \leq j \leq 150$ , it is computed the minimum distance between each data points and the initial medoids, randomly chosen. In this case, since data are in categorical format, the norm is not a suitable measure.

The most appropriate distance measure is the **Jaccard dissimilarity index**, defined in terms of the Jaccard similarity index (it is, indeed  $1 -$  the Jaccard similarity index). For two given sets, the Jaccard similarity index is defined as the ratio of the intersection of the two sets divided by their union:

$$d(x^j, c_t^{(l)}) = \min_{1 \leq l \leq k} d(x^j, c_t^{(l)})$$

$$d(x^j, c_t^{(l)}) = \frac{A \cap B}{A \cup B}$$

and assign each data points,  $x^j$ , to the corresponding (nearest) cluster. This process defines a partition,  $\Pi_t$ , for which it is possible to compute the within-cluster tightness,  $q_l(c_t^{(l)})$ , and the overall coherence,  $Q(\Pi_t)$ .

- (b) **Updating step**, in which for each cluster,  $\forall D_l$ , it is made a local distance matrix consisting of the distance between the data points of the corresponding cluster. So, it is used this matrix to interchange the medoids,  $c_t^{(l)}$ , with each data points of that cluster,  $x^j$ , in order to find the point that minimizes the within-cluster coherence: that it is an easy task since we need to find the minimum sum of the points of each column, the minimum one is the chosen. Then, the within-cluster tightness is recomputed using as medoids the data point chosen,  $q_l(x^j)$ .

If the within-cluster coherence of the data point,  $q_l(x^j)$ , is minor than the within-cluster tightness of the medoid,  $c_t^{(l)}$ , then the data point is interchange with the medoid and this applied also with the within-cluster coherence.

**If**

$$q_l(x^j) < q_l(c_t^{(l)})$$

**then**

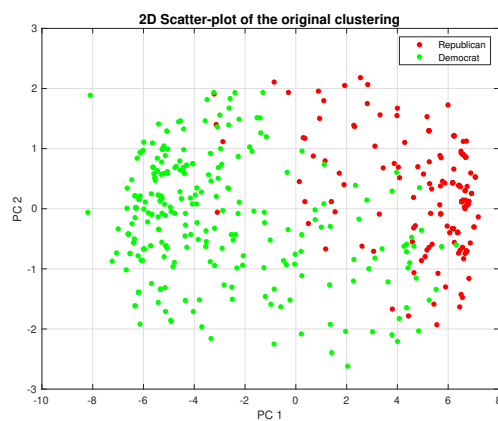
$$c_t^{(l)} = x^j, \quad q_l(c_t^{(l)}) = q_l(x^j)$$

This process produces the final medoids, that can be seen, numerically, in the table 14 and, visually, on the figure 24b and 25a, where it is also possible to see the final clustering of the data into three cluster in a bi-dimensional view i the first figure and in a three- one in the second one.

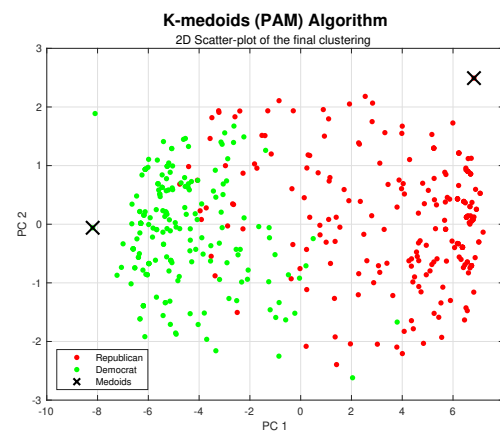
| Congressional Vote                     |            |          |
|----------------------------------------|------------|----------|
|                                        | Republican | Democrat |
| Handicapped infants                    | -1         | 0        |
| Water project cost sharing             | 0          | 0        |
| Adoption of the budget resolution      | 0          | 1        |
| Physician fee freeze                   | 0          | -1       |
| El Salvador aid                        | 0          | -1       |
| Religious groups in schools            | 0          | -1       |
| Anti-satellite test ban                | 0          | 1        |
| Aid to Nicaraguan contras              | 0          | 1        |
| Mx missile                             | 0          | 1        |
| Immigration                            | 0          | 0        |
| Synfuels corporation cutback           | 0          | 0        |
| Education spending                     | 0          | -1       |
| Superfund right to sue                 | 0          | -1       |
| Crime                                  | 1          | -1       |
| Duty free exports                      | 0          | 0        |
| Export administration act South Africa | 0          | 0        |

Table 14: Final medoids

## Congressional Vote Dataset



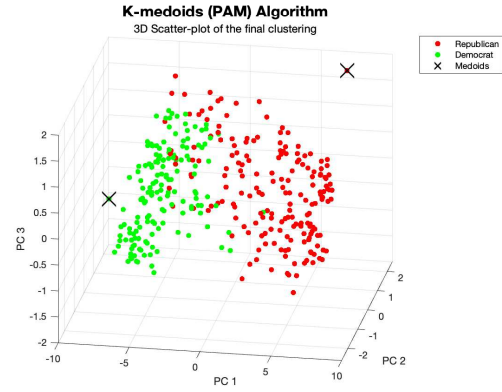
(a) Real 2D Scatter-plot



(b) Final 2D Scatter-plot

The figure 24a is a 2D scatter-plot of the first two principal components (PCs) of the Congressional Vote data matrix  $X$ , grouped by the real partitioning into cluster, according to the  $I$  column vector, contained into the Matlab data file CongressionalVote.mat.

Figure 24: a) 2D scatter-plot of the original matrix according to the  $I$  column vector. b-c) The 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the final clustering results.



(a) Final 3D Scatter-plot

4. If our criterion, in this case, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , is lower the tolerance,  $\tau$ , fixed before, the algorithm can stop; otherwise, the counter continues (it is set  $t = t + 1$ ), and the algorithm goes to the step 3.

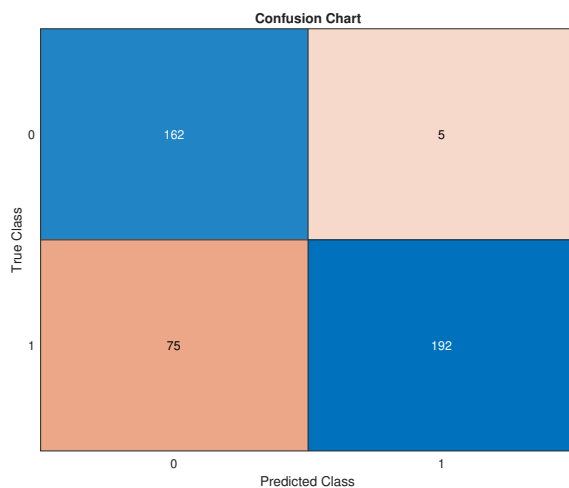
**If**

$$|Q(\Pi_t) - Q(\Pi_{t+1})| < \tau \quad \text{stop}$$

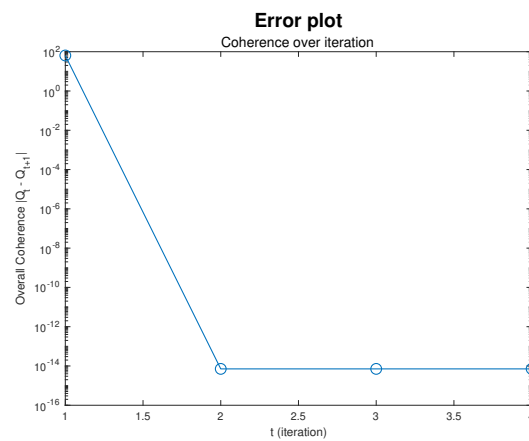
**else**

$$t = t + 1, \quad \text{go to step 3.}$$

The progress of the criterion, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , could be seen at figure 25c. It can be notice that after 2 iteration, the criterion reach 0, meaning that the overall coherence of the partitioning are equal  $Q(\Pi_t) = Q(\Pi_{t+1})$



(b) Confusion Chart



(c) Overall Coherence plot

Figure 25: a) Confusion Chart derived from the comparison of the  $I_{assign}$  (produced by my  $k$ -medoids clustering) and the  $I$  column vectors. b) Plot of the difference between the overall coherence of preceding clustering,  $Q_t$ , and of the last one,  $Q_{t+1}$ ,  $|Q_t - Q_{t+1}|$ , over the iteration,  $t$ .

The figure 25b is a visualisation of the confusion matrix (indeed, the confusion chart) in which can be seen the errors that my algorithm are made, comparing the  $I$  and my  $I\_assign$  column vector: In this case,

- the *k-means* found 162 points associated to the first cluster out of the total 167,  $162/167$ , Mistakenly assigning 5 points to the second cluster, with an error of 2.99%.
- for the second cluster, the algorithm found  $192/267$ , 192 points belong to the second cluster out of the total 267, incorrectly assigning the last 75 to first cluster, producing an error of 28.09%.

## Biopsy Dataset

The file **BiopsyDataAnnotated.mat** contains a data matrix  $X$ ,  $X \in \mathbb{R}^{9 \times 699}$ , containing breast tissue needle biopsy data from 699 patients, some of which have breast cancer, some have a benign tumor. The explanation of the columns is as follows:

$$x^j = \begin{bmatrix} \text{Clump Thickness} \\ \text{Uniformity of Cell Size} \\ \text{Uniformity of Cell Shape} \\ \text{Marginal Adhesion} \\ \text{Single Epithelial Cell Size} \\ \text{Bare Nuclei} \\ \text{Bland Chromatin} \\ \text{Normal Nucleoli} \\ \text{Mitoses} \end{bmatrix} \in \mathbb{R}^9, \quad 1 \leq j \leq 699$$

Each attribute takes on a value between 1 and 10. Some data is missing, which is indicated by a 'NaN' (= not a number) in the file. The vector  $I$  is an annotation vector,  $I \in \mathbb{R}^{1 \times 699}$ , with the annotation:

$$1 = \text{malignant} \quad 0 = \text{benign}$$

Before diving into the clustering techniques, my approach is to compute a dimensional reduction, using Principal Component Analysis (PCA), on centering data matrix,  $X_c = X - \bar{X}$ , for the only goal of better visualizing the result since we have already studied that the first Principal Component (PCs) (in the wine data-set, 2 – 3, depending on the choice of a two- or three-dimensional visualisation) are the ones that maximize the spread of the data. So, we compute the SVD of the centering data points,  $X_c = UDV'$ , for then computing the Principal Components (PCs),  $Z$ .

Our aim is to apply my *k-medoids* (PAM (Partitioning Around Medoids)) sub-routine function to the Biopsy Matrix,  $X$ :

1. First of all, fix  $k$ , the number of clusters (in this case,  $k = 2$ ) and a positive tolerance,  $\tau > 0$  (for this example,  $\tau = 1.0e^{-100}$ ), used as criterion to stop the for-loop iteration;
2. Then, the **initialisation step** in which it is chosen the initial set of medoids and the counting is started (it is set  $t = 0$ ): In this case, the initial characteristic vectors are picked randomly (with the difference that, with the respect of *k-means*, they belong to one of the data point of the cluster,  $c_0^{(l)} \in D_l$ ) ( $\{c_0^{(1)}, c_0^{(2)} | c_0^{(l)} \in \mathbb{R}^9 | 1 \leq l \leq 2\}$ ) (see Table 15). This part can be clearly seen in figure 26a, for a 2D representation, and in figure 26b, for a three-dimensional one;

| Biopsy                      |        |           |
|-----------------------------|--------|-----------|
|                             | Benign | Malignant |
| Clump Thickness             | 5      | 8         |
| Uniformity of Cell Size     | 5      | 10        |
| Uniformity of Cell Shape    | 7      | 10        |
| Marginal Adhesion           | 8      | 10        |
| Single Epithelial Cell Size | 6      | 7         |
| Bare Nuclei                 | 10     | 5         |
| Bland Chromatin             | 7      | 4         |
| Normal Nucleoli             | 4      | 8         |
| Mitoses                     | 1      | 7         |

Table 15: Initial medoids

## Biopsy Dataset

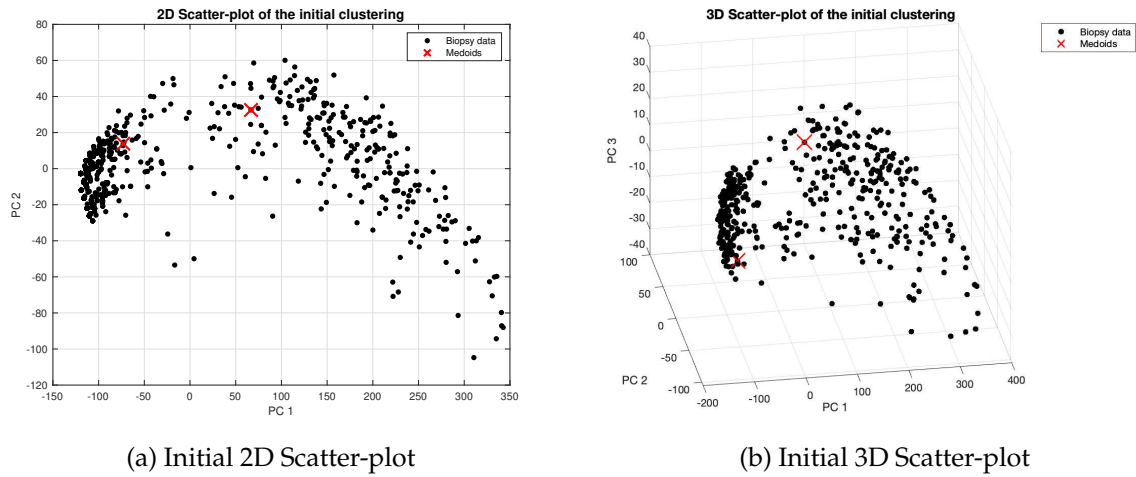


Figure 26: a-b) 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the initial seed of the characteristic vectors, the medoids,  $c^{(l)}$ , randomly chosen.

### 3. Iteration step:

- (a) **Assignment step:**  $\forall x^j, \quad 1 \leq j \leq 699$ , it is computed the minimum distance between each data points and the initial medoids, randomly chosen, (in this case, the distance measure used is the  $2norm$ )

$$d(x^j, c_t^{(l)}) = \min_{1 \leq l \leq k} d(x^j, c_t^{(l)})$$

$$d(x^j, c_t^{(l)}) = \|x^j - c_t^{(l)}\|_2$$

and assign each data points,  $x^j$ , to the corresponding (nearest) cluster. This process defines a partition,  $\Pi_t$ , for which it is possible to compute the within-cluster tightness,  $q_l(c_t^{(l)})$ , and the overall coherence,  $Q(\Pi_t)$ .



- (b) **Updating step**, in which for each cluster,  $\forall D_l$ , it is made a local distance matrix consisting of the distance between the data points of the corresponding cluster. So, it is used this matrix to interchange the medoids,  $c_t^{(l)}$ , with each data points of that cluster,  $x^j$ , in order to find the point that minimizes the within-cluster coherence: that it is an easy task since we need to find the minimum sum of the points of each column, the minimum one is the chosen. Then, the within-cluster tightness is recomputed using as medoids the data point chosen,  $q_l(x^j)$ .

If the within-cluster coherence of the data point,  $q_l(x^j)$ , is minor than the within-cluster tightness of the medoid,  $c_t^{(l)}$ , then the data point is interchange with the medoid and this applied also with the within-cluster coherence.

**If**

$$q_l(x^j) < q_l(c_t^{(l)})$$

**then**

$$c_t^{(l)} = x^j, \quad q_l(c_t^{(l)}) = q_l(x^j)$$

This process produces the final medoids, that can be seen, numerically, in the table 16 and, visually, on the figure 27a and 27b, where it is also possible to see the final clustering of the data into three cluster in a bi-dimensional view i the first figure and in a three- one in the second one.

| Biopsy                      |        |           |
|-----------------------------|--------|-----------|
|                             | Benign | Malignant |
| Clump Thickness             | 3      | 6         |
| Uniformity of Cell Size     | 1      | 6         |
| Uniformity of Cell Shape    | 1      | 6         |
| Marginal Adhesion           | 1      | 5         |
| Single Epithelial Cell Size | 2      | 4         |
| Bare Nuclei                 | 1      | 10        |
| Bland Chromatin             | 2      | 7         |
| Normal Nucleoli             | 1      | 6         |
| Mitoses                     | 1      | 2         |

Table 16: Final medoids

Biopsy Dataset

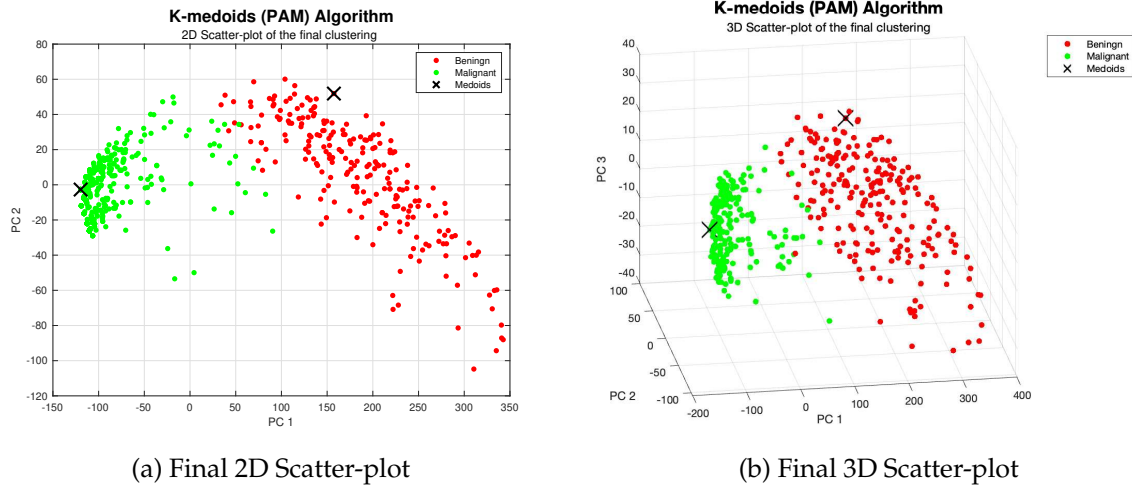


Figure 27: a-b) The 2D and 3D scatter-plot of the, respectively, 2-3 Principal Components (PCs) with the final clustering results.

4. If our criterion, in this case, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , is lower the tolerance,  $\tau$ , fixed before, the algorithm can stop; otherwise, the counter continues (it is set  $t = t + 1$ ), and the algorithm goes to the step 3.

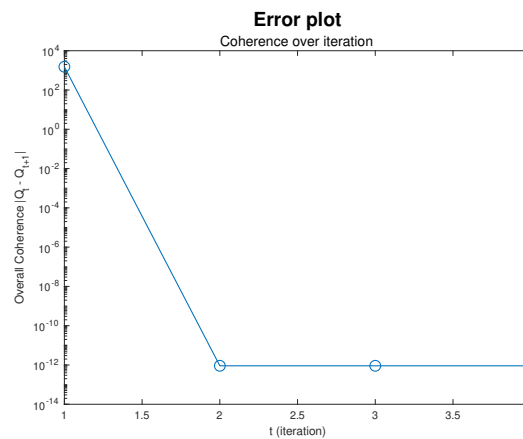
**If**

$$|Q(\Pi_t) - Q(\Pi_{t+1})| < \tau \quad \text{stop}$$

**else**

$$t = t + 1, \quad \text{go to step 3.}$$

The progress of the criterion, the absolute value of the difference of the overall coherence,  $Q$ , between the old,  $\Pi_t$ , and the new partition,  $\Pi_{t+1}$ , could be seen at figure 22c. It can be notice that after 2 iteration, the criterion reach 0, meaning that the overall coherence of the partitioning are equal  $Q(\Pi_t) = Q(\Pi_{t+1})$



(a) Overall Coherence plot

Figure 28: Plot of the difference between the overall coherence of preceding clustering,  $Q_t$ , and of the last one,  $Q_{t+1}$ ,  $|Q_t - Q_{t+1}|$ , over the iteration,  $t$ .

# Appendices

## Appendix

### Function.

#### *K-means.*

```

1 function [D, I_assign, Dnew, centroids, initial_means] = mykm(X, k, tol
2 )
3 % rng(1);
4
5 [n_row, n_col] = size(X); %where i is the number of rows (variable) and
6   j is the number of columns (obsrvations)
7
8 %initial partitioning
9 init_seed = X(:,randperm(n_col, k));
10
11 len_par = floor(n_col/k);
12 D = cell(1, k);
13 for h = 1:k
14     D{h} = X(:, ((h-1)*len_par + 1):(h*len_par));
15
16     ql(h) = sum(norm((D{h} - init_seed(:,h)), 2)); %WSS k clusters
17 end
18 Q = sum(ql, 2); %overall coherence of the clustering
19
20 initial_means = init_seed; %I need the former init_seed for
21   visualisation.
22
23 Dnew = cell(1, k);
24 centroids = zeros(n_row, k);
25 Dist = zeros(k, n_col);
26 I_assign = zeros(1, n_col);
27 min_values = zeros(1, n_col);
28
29 crit = 1;
30 maxiteration = 100000;
31 t = 0;
32 while (t < maxiteration)
33     for h = 1:k
34         for j = 1:n_col
35             Dist(h, j) = norm(X(:, j) - init_seed(:,h), 2); %distance
36             all data points - centroid
37         end
38     end
39
40     [min_values, I_assign] = min(Dist, [], 1); %minimum between each
41     value
42
43     for h = 1:k
44         % Assign data points to the corresponding clusters
45         Dnew{h} = X(:, I_assign == h);
46     end
47 end

```

```

45     centroids(:, h) = mean(Dnew{h}, 2); %compute centroids as mean
      of each cluster
46
47     ql_Dnew(h) = sum(vecnorm((Dnew{h} - centroids(:, h)), 2, 1)); %
WSS new h clusters
48     end
49
50     Qnew = sum(ql_Dnew); %overall coherence of the new clustering
51     crit = abs(Q - Qnew); %criterium
52
53     Err(t+1) = crit;
54
55     if (crit < tol)
56         break;
57     else
58         init_seed = centroids;
59         Q = Qnew;
60     end
61
62     t = t+1;
63 end
64
65 %coherence over iteration plot
66 figure;
67 semilogy(Err, 'o-', 'MarkerSize', 10, 'LineWidth', 1)
68 title('Error plot', 'FontSize', 18);
69 subtitle('Coherence over iteration', 'FontSize', 13);
70 xlabel('t (iteration)', 'FontSize', 11);
71 ylabel('Overall Coherence |Q_t - Q_{t+1}|', 'FontSize', 11);

```

### K-medoids.

```

1
2 function [I_assign, I_m, starting_medoids] = my_k_medoids(D, k, tol)
3
4 %rng(1); %set seed for replicability
5 p = size(D, 1);
6 I_m = sort(randperm(p, k)); %pick up 3 random ind
7
8 starting_medoids = I_m; %starting_medoids indeces
9
10
11 crit = 1;
12 maxiteration = 100000;
13 t = 0;
14 while (t < maxiteration)
15
16     %% assignment step
17
18     D_m = D(:, I_m); %distances w.r.t. medoids submatrix
19
20     [q, I_assign] = min(D_m'); %TRANPOSE! %index to clusters
21
22     Q = sum(q); %Efficient strategy for PAM
23
24     oldI_m = I_m;
25

```

```

26
27 %% updating step
28 for h = 1:k
29
30     I_k = find(I_assign == h); % Indices to points in the cluster
31
32     D_k = D(I_k, I_k);
33
34     [ql_new(h), j] = min(sum(D_k));
35
36
37     %from local to global indices
38     I_m(h) = I_k(j); %swap (updating) of the medoids indeces
39
40 end
41
42 %% recompute the global coherence
43 Qnew = sum(ql_new);
44
45
46 %% if not converged, continue from the assignment step
47 crit = abs(Q - Qnew);
48
49 Err(t+1) = crit;
50
51 t = t+1;
52
53 if (crit < tol)
54     break
55 else
56     Q = Qnew;
57 end
58 end
59
60 final_medoids = I_m;
61
62 figure;
63 semilogy(Err, 'o-', 'MarkerSize', 10, 'LineWidth', 1)
64 xlim([1 5])
65 title('Error plot', 'FontSize', 18);
66 subtitle('Coherence over iteration', 'FontSize', 13);
67 xlabel('t (iteration)', 'FontSize', 11);
68 ylabel('Overall Coherence |Q_t - Q_{t+1}|', 'FontSize', 11);

```

## Iris Dataset.

### K-means.

```

1 %% Iris Dataset - k-means
2 clear all
3 close all
4 load IrisDataAnnotated.mat
5
6
7 %k_means
8 [D, I_assign, Dnew, centroids, initial_means] = mykm(X, 3, 1.0e-100);

```

```

9
10 %PCA
11 X_cen = X - mean(X, 2);
12 [U,~,~] = svd(X_cen);
13 Z = U' * X_cen;
14
15 for h = 1:3
16     Orig{h} = Z(:, I == h);
17 end
18
19 %Viusalisation
20 %2d scatter-plot of the original partitioning
21 figure;
22 plot(Orig{1}(1, :), Orig{1}(2, :), 'r.', 'MarkerSize', 15); % Cluster 1
    (red)
23 hold on;
24 plot(Orig{2}(1, :), Orig{2}(2, :), 'g.', 'MarkerSize', 15); % Cluster 2
    (green)
25 plot(Orig{3}(1, :), Orig{3}(2, :), 'b.', 'MarkerSize', 15); % Cluster 3
    (blue)
26 xlabel('PC 1', 'FontSize', 11);
27 ylabel('PC 2', 'FontSize', 11);
28 title('2D Scatter Plot of the original Clustering', 'FontSize', 13);
29 legend('Setosa', 'Versacolor', 'Virginica');
30 grid on;
31 %2d scatter-plot of the initial partitioning
32 figure;
33 plot(D{1}(1, :), D{1}(2, :), 'r.', 'MarkerSize', 15); % Cluster 1 (red)
34 hold on;
35 plot(D{2}(1, :), D{2}(2, :), 'g.', 'MarkerSize', 15); % Cluster 2 (
    green)
36 plot(D{3}(1, :), D{3}(2, :), 'b.', 'MarkerSize', 15); % Cluster 3 (blue
    )
37 plot(initial_means(1, 1), initial_means(2, 1), 'kx', 'MarkerSize', 18,
    'LineWidth', 2); % Centroid for cluster 1
38 plot(initial_means(1, 2), initial_means(2, 2), 'kx', 'MarkerSize', 18,
    'LineWidth', 2); % Centroid for cluster 2
39 plot(initial_means(1, 3), initial_means(2, 3), 'kx', 'MarkerSize', 18,
    'LineWidth', 2); % Centroid for cluster 3
40 xlabel('PC 1', 'FontSize', 11);
41 ylabel('PC 2', 'FontSize', 11);
42 title('2D Scatter Plot of the initial Clustering', 'FontSize', 13);
43 legend('Setosa', 'Versacolor', 'Virginica', 'Initial seed');
44 grid on;
45
46 %2d scatter-plot of the new partitioning
47 figure;
48 plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15); % Cluster 1
    (red)
49 hold on;
50 plot(Dnew{2}(1, :), Dnew{2}(2, :), 'g.', 'MarkerSize', 15); % Cluster 2
    (green)
51 plot(Dnew{3}(1, :), Dnew{3}(2, :), 'b.', 'MarkerSize', 15); % Cluster 3
    (blue)
52 plot(centroids(1, 1), centroids(2, 1), 'kx', 'MarkerSize', 18, '
    LineWidth', 2); % Centroid for cluster 1
53 plot(centroids(1, 2), centroids(2, 2), 'kx', 'MarkerSize', 18, '
    LineWidth', 2); % Centroid for cluster 2

```

```

54 plot(centroids(1, 3), centroids(2, 3), 'kx', 'MarkerSize', 18, '
    LineWidth', 2); % Centroid for cluster 3
55 xlabel('PC 1', 'FontSize', 11);
56 ylabel('PC 2', 'FontSize', 11);
57 title('K-means (Lloyd) Algorithm', 'FontSize', 16);
58 subtitle('2D Scatter Plot of the final Clustering', 'FontSize', 11.5);
59 legend('Setosa', 'Versicolor', 'Virginica', 'Centroids');
60 grid on;
61
62 %Voronoi Tessellation
63 figure;
64 [vx, vy] = voronoi(centroids(1, :), centroids(2, :));
65 plot(vx, vy, 'k-');
66 hold on
67 h1 = plot(centroids(1, 1), centroids(2, 1), 'ro', 'MarkerSize', 10, '
    LineWidth', 2.5); % Centroid for cluster 1
68 h2 = plot(centroids(1, 2), centroids(2, 2), 'go', 'MarkerSize', 10, '
    LineWidth', 2); % Centroid for cluster 2
69 h3 = plot(centroids(1, 3), centroids(2, 3), 'bo', 'MarkerSize', 10, '
    LineWidth', 2); % Centroid for cluster 3
70 xlabel('PC 1', 'FontSize', 11);
71 ylabel('PC 2', 'FontSize', 11);
72 title('K-means with Voronoi Tessellation', 'FontSize', 13);
73 legend([h1, h2, h3], {'Setosa Centroid', 'Versicolor Centroid', '
    Virginica Centroid'})
74 xlim([4.36 6.95]);
75 ylim([1.24 6.57]);
76 grid on;
77 hold off
78
79 %2D Scatterplot of the final clustering with Voronoi Tessellation
80 figure;
81 h1 = plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15); %
    Cluster 1 (red)
82 hold on;
83 h2 = plot(Dnew{2}(1, :), Dnew{2}(2, :), 'g.', 'MarkerSize', 15); %
    Cluster 2 (green)
84 h3 = plot(Dnew{3}(1, :), Dnew{3}(2, :), 'b.', 'MarkerSize', 15); %
    Cluster 3 (blue)
85 [vx, vy] = voronoi(centroids(1, :), centroids(2, :));
86 plot(vx, vy, 'k-');
87 h4 = plot(centroids(1, 1), centroids(2, 1), 'ko', 'MarkerSize', 10, '
    LineWidth', 2.5); % Centroid for cluster 1
88 plot(centroids(1, 2), centroids(2, 2), 'ko', 'MarkerSize', 10, '
    LineWidth', 2); % Centroid for cluster 2
89 plot(centroids(1, 3), centroids(2, 3), 'ko', 'MarkerSize', 10, '
    LineWidth', 2); % Centroid for cluster 3
90 xlabel('PC 1', 'FontSize', 11);
91 ylabel('PC 2', 'FontSize', 11);
92 title('K-means with Voronoi Tessellation', 'FontSize', 16);
93 subtitle('2D Scatter Plot of the final Clustering', 'FontSize', 11.5);
94 legend([h1, h2, h3, h4], {'Setosa', 'Versicolor', 'Virginica', '
    Centroids'})
95 xlim([4.36 6.95]);
96 ylim([1.24 6.57]);
97 grid on;
98 hold off
99

```

```

100 %3d scatter-plot of the initial partitioning
101 figure;
102 plot3(D{1}(1, :), D{1}(2, :), D{1}(3, :), 'r.', 'MarkerSize', 15); %
    Cluster 1 (red)
103 hold on;
104 plot3(D{2}(1, :), D{2}(2, :), D{2}(3, :), 'g.', 'MarkerSize', 15); %
    Cluster 2 (green)
105 plot3(D{3}(1, :), D{3}(2, :), D{3}(3, :), 'b.', 'MarkerSize', 15); %
    Cluster 3 (blue)
106 plot3(initial_means(1, 1), initial_means(2, 1), initial_means(3, 1), '
    kx', 'MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 1
107 plot3(initial_means(1, 2), initial_means(2, 2), initial_means(3, 2), 'kx
    ', 'MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 2
108 plot3(initial_means(1, 3), initial_means(2, 3), initial_means(3, 3), 'kx
    ', 'MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 3
109 xlabel('PC 1', 'FontSize', 11);
110 ylabel('PC 2', 'FontSize', 11);
111 zlabel('PC 3', 'FontSize', 11);
112 title('3D Scatter Plot of the initial Clustering', 'FontSize', 13);
113 legend('Setosa', 'Versicolor', 'Virginica', 'Initial seed');
114 grid on;
115
116 %3d scatter-plot of the new partitioning
117 figure;
118 plot3(Dnew{1}(1,:), Dnew{1}(2, :), Dnew{1}(3, :), 'r.', 'MarkerSize',
    15); % Cluster 1 (red)
119 hold on;
120 plot3(Dnew{2}(1, :), Dnew{2}(2, :), Dnew{2}(3, :), 'g.', 'MarkerSize',
    15); % Cluster 2 (green)
121 plot3(Dnew{3}(1, :), Dnew{3}(2, :), Dnew{3}(3, :), 'b.', 'MarkerSize',
    15); % Cluster 3 (blue)
122 plot3(centroids(1, 1), centroids(2, 1), centroids(3, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 1
123 plot3(centroids(1, 2), centroids(2, 2), centroids(3, 2), 'kx', '
    MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 2
124 plot3(centroids(1, 3), centroids(2, 3), centroids(3, 3), 'kx', '
    MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 3
125 xlabel('PC 1', 'FontSize', 11);
126 ylabel('PC 2', 'FontSize', 11);
127 zlabel('PC 3', 'FontSize', 11);
128 title('K-means (Lloyd) Algorithm', 'FontSize', 16);
129 subtitle('3D Scatter-Plot of the final Clustering', 'FontSize', 11.5);
130 legend('Setosa', 'Versicolor', 'Virginica', 'Centroids');
131 grid on;
132
133 %Confusion Chart
134 figure;
135 cm = confusionchart(I, I_assign);
136 title('Confusion Chart');

```

## K-medoids.

### 1-norm

```

1 %% Iris Dataset - k-medoids
2

```



```

3 clear all
4 close all
5 load IrisDataAnnotated.mat
6
7 %distance matrix
8 [n, p] = size(X);
9 for i = 1:p
10     for j = i:p
11         if (i == j)
12             D(i, j) = 0;
13         else
14             D(i, j) = norm(X(:, i) - X(:, j), 1);
15             %TRY ALSO NORM 1 and NORM 'inf'
16             %compare also the iteration number
17         end
18     end
19 %     end
20 end
21 D = D+D';
22
23 %k-medoids
24 [I_assign, I_m, starting_medoids] = my_k_medoids(D, 3, 1.0e-30);
25
26 %PCA
27 X_cen = X - mean(X, 2);
28 [U,~,~] = svd(X_cen);
29 Z = U' * X_cen;
30
31 %clustering
32 for j = 1:3
33     Dnew{j} = Z(:, find(I_assign == j)); %k-ith cluster
34     X_final_medoids{j} = Z(:, I_m(j));
35     X_init_medoids{j} = Z(:, starting_medoids(j));
36
37     Orig{j} = Z(:, find(I == j)); %original clustering
38 end
39
40 %visualisation
41 %2D scatterplot of the initial clustering
42 figure;
43 plot(Z(1, :), Z(2, :), 'k.', 'MarkerSize', 15)
44 hold on
45 plot(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
46 hold on
47 plot(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
48 hold on
49 plot(X_init_medoids{3}(1, 1), X_init_medoids{3}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
50 xlabel('Iris flower 1', 'FontSize', 11);
51 ylabel('Iris flower 2', 'FontSize', 11);
52 legend('Iris', 'Medoids');
53 title('2D Scatter-plot of the initial clustering', 'FontSize', 13);
54 grid on;
55
56 %3D scatterplot of the intial clustering
57 figure;

```

```

58 plot3(Z(1, :), Z(2, :), Z(3, :), 'k.', 'MarkerSize', 15)
59 hold on
60 plot3(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), X_init_medoids
    {1}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
61 hold on
62 plot3(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), X_init_medoids
    {2}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
63 hold on
64 plot3(X_init_medoids{3}(1, 1), X_init_medoids{3}(2, 1), X_init_medoids
    {3}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
65 xlabel('Iris flower 1', 'FontSize', 11);
66 ylabel('Iris flower 2', 'FontSize', 11);
67 zlabel('Iris flower 3', 'FontSize', 11);
68 legend('Iris', 'Medoids');
69 title('3D Scatter-plot of the initial clustering', 'FontSize', 13);
70 grid on;
71
72 %2D scatterplot of the original clustering
73 figure;
74 plot(Orig{1}(1, :), Orig{1}(2, :), 'r.', 'MarkerSize', 15)
75 hold on
76 plot(Orig{2}(1, :), Orig{2}(2, :), 'b.', 'MarkerSize', 15)
77 hold on
78 plot(Orig{3}(1, :), Orig{3}(2, :), 'g.', 'MarkerSize', 15)
79 xlabel('Iris flower 1', 'FontSize', 11);
80 ylabel('Iris flower 2', 'FontSize', 11);
81 legend('Setosa', 'Versicolor', 'Virginica');
82 title('2D Scatter-plot of the original clustering', 'FontSize', 13);
83 grid on;
84
85 %2D scatterplot of the final clustering
86 figure;
87 plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15)
88 hold on
89 plot(Dnew{2}(1, :), Dnew{2}(2, :), 'b.', 'MarkerSize', 15)
90 hold on
91 plot(Dnew{3}(1, :), Dnew{3}(2, :), 'g.', 'MarkerSize', 15)
92 hold on
93 plot(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
94 hold on
95 plot(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
96 hold on
97 plot(X_final_medoids{3}(1, 1), X_final_medoids{3}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
98 xlabel('Iris flower 1', 'FontSize', 11);
99 ylabel('Iris flower 2', 'FontSize', 11);
100 legend('Setosa', 'Versicolor', 'Virginica', 'Medoids');
101 title('K-medoids (PAM) Algorithm', 'FontSize', 16)
102 subtitle('2D Scatter-plot of the final clustering', 'FontSize', 11.5);
103 grid on;
104
105 %3D Scatter-plot of the final clustering
106 figure;
107 plot3(Dnew{1}(1, :), Dnew{1}(2, :), Dnew{1}(3, :), 'r.', 'MarkerSize',
    15)
108 hold on

```

```

109 plot3(Dnew{2}(1, :), Dnew{2}(2, :), Dnew{2}(3, :), 'b.', 'MarkerSize',
      15)
110 hold on
111 plot3(Dnew{3}(1, :), Dnew{3}(2, :), Dnew{3}(3, :), 'g.', 'MarkerSize',
      15)
112 hold on
113 plot3(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1),
      X_final_medoids{1}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
114 hold on
115 plot3(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1),
      X_final_medoids{2}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
116 hold on
117 plot3(X_final_medoids{3}(1, 1), X_final_medoids{3}(2, 1),
      X_final_medoids{3}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
118 xlabel('Iris flower 1', 'FontSize', 11);
119 ylabel('Iris flower 2', 'FontSize', 11);
120 zlabel('Iris flower 3', 'FontSize', 11);
121 legend('Setosa', 'Versicolor', 'Virginica', 'Medoids');
122 title('K-medoids (PAM) Algorithm', 'FontSize', 16)
123 subtitle('3D Scatter-plot of the final clustering', 'FontSize', 11.5);
124 grid on;
125
126 %confusion matrix
127 figure;
128 cm_1 = confusionchart(I, I_assign);
129 title('Confusion Chart')

```

## 2-norm

```

1
2 %% Iris Dataset - K-medoids
3 clear all
4 close all
5 load IrisDataAnnotated.mat
6
7 %distance matrix
8 [n,p] = size(X);
9 for i = 1:p
10     for j = i:p
11         if (i == j)
12             D(i,j) = 0;
13         else
14             D(i,j) = norm(X(:, i) - X(:, j), 2);
15             %TRY ALSO NORM 1 and NORM 'inf'
16             %compare also the iteration number
17         end
18     end
19 %     end
20 end
21 D = D+D';
22
23 %k-medoids
24 [I_assign, I_m, starting_medoids] = my_k_medoids(D, 3, 1.0e-100);
25
26 %PCA
27 X_cen = X - mean(X, 2);
28 [U,~,~] = svd(X_cen);

```

```

29 Z = U' * X_cen;
30
31 %clustering
32 for j = 1:3
33     Dnew{j} = Z(:, find(I_assign == j)); %k-ith cluster
34     X_final_medoids{j} = Z(:, I_m(j));
35     X_init_medoids{j} = Z(:, starting_medoids(j));
36
37     Orig{j} = Z(:, find(I == j)); %original clustering
38 end
39
40 %visualisation
41 %2D scatterplot of the initial clustering
42 figure;
43 plot(Z(1, :), Z(2, :), 'k.', 'MarkerSize', 15)
44 hold on
45 plot(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
46 hold on
47 plot(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
48 hold on
49 plot(X_init_medoids{3}(1, 1), X_init_medoids{3}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
50 xlabel('PC 1', 'FontSize', 11);
51 ylabel('PC 2', 'FontSize', 11);
52 legend('Iris', 'Medoids');
53 title('2D Scatter-plot of the initial clustering', 'FontSize', 13);
54 grid on;
55
56 %3D scatterplot of the initial clustering
57 figure;
58 plot3(Z(1, :), Z(2, :), Z(3, :), 'k.', 'MarkerSize', 15)
59 hold on
60 plot3(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), X_init_medoids
    {1}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
61 hold on
62 plot3(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), X_init_medoids
    {2}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
63 hold on
64 plot3(X_init_medoids{3}(1, 1), X_init_medoids{3}(2, 1), X_init_medoids
    {3}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
65 xlabel('PC 1', 'FontSize', 11);
66 ylabel('PC 2', 'FontSize', 11);
67 zlabel('PC 3', 'FontSize', 11);
68 legend('Iris', 'Medoids');
69 title('3D Scatter-plot of the initial clustering', 'FontSize', 13);
70 grid on;
71
72 %2D scatterplot of the original clustering
73 figure;
74 plot(Orig{1}(1, :), Orig{1}(2, :), 'r.', 'MarkerSize', 15)
75 hold on
76 plot(Orig{2}(1, :), Orig{2}(2, :), 'b.', 'MarkerSize', 15)
77 hold on
78 plot(Orig{3}(1, :), Orig{3}(2, :), 'g.', 'MarkerSize', 15)
79 xlabel('PC 1', 'FontSize', 11);
80 ylabel('PC 2', 'FontSize', 11);

```

```

81 legend('Setosa', 'Versicolor', 'Virginica');
82 title('2D Scatter-plot of the original clustering', 'FontSize', 13);
83 grid on;
84
85 %2D Scatterplot of the final clustering
86 figure;
87 plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15)
88 hold on
89 plot(Dnew{2}(1, :), Dnew{2}(2, :), 'b.', 'MarkerSize', 15)
90 hold on
91 plot(Dnew{3}(1, :), Dnew{3}(2, :), 'g.', 'MarkerSize', 15)
92 hold on
93 plot(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
94 hold on
95 plot(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
96 hold on
97 plot(X_final_medoids{3}(1, 1), X_final_medoids{3}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
98 xlabel('PC 1', 'FontSize', 11);
99 ylabel('PC 2', 'FontSize', 11);
100 legend('Setosa', 'Versicolor', 'Virginica', 'Medoids');
101 title('K-medoids (PAM) Algorithm', 'FontSize', 16);
102 subtitle('2D Scatter-plot of the final clustering', 'FontSize', 11.5);
103 grid on;
104
105 %3D Scatter-plot of the final clustering
106 figure;
107 plot3(Dnew{1}(1, :), Dnew{1}(2, :), Dnew{1}(3, :), 'r.', 'MarkerSize',
    15)
108 hold on
109 plot3(Dnew{2}(1, :), Dnew{2}(2, :), Dnew{2}(3, :), 'b.', 'MarkerSize',
    15)
110 hold on
111 plot3(Dnew{3}(1, :), Dnew{3}(2, :), Dnew{3}(3, :), 'g.', 'MarkerSize',
    15)
112 hold on
113 plot3(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1),
    X_final_medoids{1}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
114 hold on
115 plot3(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1),
    X_final_medoids{2}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
116 hold on
117 plot3(X_final_medoids{3}(1, 1), X_final_medoids{3}(2, 1),
    X_final_medoids{3}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
118 xlabel('PC 1', 'FontSize', 11);
119 ylabel('PC 2', 'FontSize', 11);
120 zlabel('PC 3', 'FontSize', 11);
121 legend('Setosa', 'Versicolor', 'Virginica', 'Medoids');
122 title('K-medoids (PAM) Algorithm', 'FontSize', 16);
123 subtitle('3D Scatter-plot of the final clustering', 'FontSize', 11.5);
124 grid on;
125
126 %Confusion Chart
127 figure;
128 cm_1 = confusionchart(I, I_assign);
129 title('Confusion Chart');

```

## Wine Dataset.

### K-means.

```

1 %% Wine Dataset - k-means
2 clear all
3 close all
4 load WineData.mat
5
6 %PCA
7 X_cen = X - mean(X, 2);
8 [U,~,~] = svd(X_cen);
9 Z = U' * X_cen;
10
11 %k_means
12 [D, I_assign, Dnew, centroids, initial_means] = mykm(Z, 3, 1.0e-100);
13
14 for h = 1:3
15     Orig{h} = Z(:, I == h);
16 end
17
18 %Viusalisation
19 %2D scatter-plot of the original partitioning
20 figure;
21 plot(Orig{1}(1, :), Orig{1}(2, :), 'r.', 'MarkerSize', 15); % Cluster 1
    (red)
22 hold on;
23 plot(Orig{2}(1, :), Orig{2}(2, :), 'g.', 'MarkerSize', 15); % Cluster 2
    (green)
24 plot(Orig{3}(1, :), Orig{3}(2, :), 'b.', 'MarkerSize', 15); % Cluster 3
    (blue)
25 xlabel('PC 1', 'FontSize', 11);
26 ylabel('PC 2', 'FontSize', 11);
27 title('2D Scatter Plot of the original Clustering', 'FontSize', 13);
28 legend('Setosa', 'Versacolor', 'Virginica');
29 grid on;
30
31 %2D scatter-plot of the initial partitioning
32 figure;
33 plot(D{1}(1, :), D{1}(2, :), 'r.', 'MarkerSize', 15); % Cluster 1 (red)
34 hold on;
35 plot(D{2}(1, :), D{2}(2, :), 'g.', 'MarkerSize', 15); % Cluster 2 (
    green)
36 plot(D{3}(1, :), D{3}(2, :), 'b.', 'MarkerSize', 15); % Cluster 3 (blue
    )
37 plot(initial_means(1, 1), initial_means(2, 1), 'kx', 'MarkerSize', 18,
    'LineWidth', 2); % Centroid for cluster 1
38 plot(initial_means(1, 2), initial_means(2, 2), 'kx', 'MarkerSize', 18,
    'LineWidth', 2); % Centroid for cluster 2
39 plot(initial_means(1, 3), initial_means(2, 3), 'kx', 'MarkerSize', 18,
    'LineWidth', 2); % Centroid for cluster 3
40 xlabel('PC 1', 'FontSize', 11);
41 ylabel('PC 2', 'FontSize', 11);
42 title('2D Scatter Plot of the initial Clustering', 'FontSize', 13);

```

```

43 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3', 'Initial seed');
44 grid on;
45
46 %2D scatter-plot of the new partitioning
47 figure;
48 plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15); % Cluster 1
    (red)
49 hold on;
50 plot(Dnew{2}(1, :), Dnew{2}(2, :), 'g.', 'MarkerSize', 15); % Cluster 2
    (green)
51 plot(Dnew{3}(1, :), Dnew{3}(2, :), 'b.', 'MarkerSize', 15); % Cluster 3
    (blue)
52 plot(centroids(1, 1), centroids(2, 1), 'kx', 'MarkerSize', 18, '
    LineWidth', 2); % Centroid for cluster 1
53 plot(centroids(1, 2), centroids(2, 2), 'kx', 'MarkerSize', 18, '
    LineWidth', 2); % Centroid for cluster 2
54 plot(centroids(1, 3), centroids(2, 3), 'kx', 'MarkerSize', 18, '
    LineWidth', 2); % Centroid for cluster 3
55 xlabel('PC 1', 'FontSize', 11);
56 ylabel('PC 2', 'FontSize', 11);
57 title('K-means (Lloyd) Algorithm', 'FontSize', 16);
58 subtitle('2D Scatter Plot of the final Clustering', 'FontSize', 11.5);
59 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3', 'Centroids');
60 grid on;
61
62 %Voronoi Tessellation
63 figure;
64 [vx, vy] = voronoi(centroids(1, 1:3), centroids(2, 1:3));
65 plot(vx, vy, 'k-');
66 hold on
67 h1 = plot(centroids(1, 1), centroids(2, 1), 'ro', 'MarkerSize', 10, '
    LineWidth', 2.5); % Centroid for cluster 1
68 h2 = plot(centroids(1, 2), centroids(2, 2), 'go', 'MarkerSize', 10, '
    LineWidth', 2); % Centroid for cluster 2
69 h3 = plot(centroids(1, 3), centroids(2, 3), 'bo', 'MarkerSize', 10, '
    LineWidth', 2); % Centroid for cluster 3
70 xlabel('PC 1', 'FontSize', 11);
71 ylabel('PC 2', 'FontSize', 11);
72 title('K-means with Voronoi Tessellation', 'FontSize', 13);
73 legend([h1, h2, h3], {'Cultivar 1 Centroid', 'Cultivar 2 Centroid', '
    Cultivar 3 Centroid'})
74 xlim([-700 500]);
75 ylim([-850 18000]);
76 grid on;
77 hold off
78
79 %2D Scatterplot of the final clustering with Voronoi Tessellation
80 figure;
81 h1 = plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15); %
    Cluster 1 (red)
82 hold on;
83 h2 = plot(Dnew{2}(1, :), Dnew{2}(2, :), 'g.', 'MarkerSize', 15); %
    Cluster 2 (green)
84 h3 = plot(Dnew{3}(1, :), Dnew{3}(2, :), 'b.', 'MarkerSize', 15); %
    Cluster 3 (blue)
85 [vx, vy] = voronoi(centroids(1, :), centroids(2, :));
86 plot(vx, vy, 'k-');

```

```

87 h4 = plot(centroids(1, 1), centroids(2, 1), 'ko', 'MarkerSize', 10, '
    LineWidth', 2.5); % Centroid for cluster 1
88 plot(centroids(1, 2), centroids(2, 2), 'ko', 'MarkerSize', 10, '
    LineWidth', 2); % Centroid for cluster 2
89 plot(centroids(1, 3), centroids(2, 3), 'ko', 'MarkerSize', 10, '
    LineWidth', 2); % Centroid for cluster 3
90 xlabel('PC 1', 'FontSize', 11);
91 ylabel('PC 2', 'FontSize', 11);
92 title('K-means with Voronoi Tessellation', 'FontSize', 16);
93 subtitle('2D Scatter Plot of the final Clustering', 'FontSize', 11.5);
94 legend([h1, h2, h3, h4], {'Cultivar 1 centroid', 'Cultivar 2 centroid',
    'Cultivar 3 centroid', 'Centroids'})
95 xlim([-700 500]);
96 ylim([-850 18000]);
97 grid on;
98 hold off
99
100 %3D scatter-plot of the initial partitioning
101 figure;
102 plot3(D{1}(1, :), D{1}(2, :), D{1}(3, :), 'r.', 'MarkerSize', 15); %
    Cluster 1 (red)
103 hold on;
104 plot3(D{2}(1, :), D{2}(2, :), D{2}(3, :), 'g.', 'MarkerSize', 15); %
    Cluster 2 (green)
105 plot3(D{3}(1, :), D{3}(2, :), D{3}(3, :), 'b.', 'MarkerSize', 15); %
    Cluster 3 (blue)
106 plot3(initial_means(1, 1), initial_means(2, 1), initial_means(3, 1), '
    kx', 'MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 1
107 plot3(initial_means(1, 2), initial_means(2, 2), initial_means(3, 2), 'kx
    ', 'MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 2
108 plot3(initial_means(1, 3), initial_means(2, 3), initial_means(3, 3), 'kx
    ', 'MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 3
109 xlabel('PC 1', 'FontSize', 11);
110 ylabel('PC 2', 'FontSize', 11);
111 zlabel('PC 3', 'FontSize', 11);
112 title('3D Scatter Plot of the initial Clustering', 'FontSize', 13);
113 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3', 'Initial seed');
114 grid on;
115
116 %3D scatter-plot of the new partitioning
117 figure;
118 plot3(Dnew{1}(1,:), Dnew{1}(2, :), Dnew{1}(3, :), 'r.', 'MarkerSize',
    15); % Cluster 1 (red)
119 hold on;
120 plot3(Dnew{2}(1, :), Dnew{2}(2, :), Dnew{2}(3, :), 'g.', 'MarkerSize',
    15); % Cluster 2 (green)
121 plot3(Dnew{3}(1, :), Dnew{3}(2, :), Dnew{3}(3, :), 'b.', 'MarkerSize',
    15); % Cluster 3 (blue)
122 plot3(centroids(1, 1), centroids(2, 1), centroids(3, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 1
123 plot3(centroids(1, 2), centroids(2, 2), centroids(3, 2), 'kx', '
    MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 2
124 plot3(centroids(1, 3), centroids(2, 3), centroids(3, 3), 'kx', '
    MarkerSize', 18, 'LineWidth', 2); % Centroid for cluster 3
125 xlabel('PC 1', 'FontSize', 11);
126 ylabel('PC 2', 'FontSize', 11);
127 zlabel('PC 3', 'FontSize', 11);
128 title('K-means (Lloyd) Algorithm', 'FontSize', 16);

```



```

129 subtitle('3D Scatter-Plot of the new Clustering', 'FontSize', 11.5);
130 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3', 'Centroids');
131 grid on;
132
133
134 %Confusion Chart
135 figure;
136 cm = confusionchart(I, I_assign);
137 title('Confusion Chart')

```

### *K-medoids.*

```

1 %% Wine Dataset - k-medoids
2 clear all
3 close all
4 load WineData.mat
5
6 %Distance matrix
7 [n, p] = size(X);
8 for i = 1:p
9     for j = i:p
10        if (i == j)
11            D(i, j) = 0;
12        else
13            D(i, j) = norm(X(:, i) - X(:, j), 2);
14            %TRY ALSO NORM 1 and NORM 'inf'
15            %compare also the iteration number
16        end
17    end
18 %    end
19 end
20 D = D+D';
21
22 %k-medoids
23 [I_assign, I_m, starting_medoids] = my_k_medoids(D, 3, 1.0e-100);
24
25 %PCA
26 X_cen = X - mean(X, 2);
27 [U,~,~] = svd(X_cen);
28 Z = U' * X_cen;
29
30 %Clustering
31 for j = 1:3
32     Dnew{j} = Z(:, find(I_assign == j)); %k-ith cluster
33     X_final_medoids{j} = Z(:, I_m(j));
34     X_init_medoids{j} = Z(:, starting_medoids(j));
35
36     Orig{j} = Z(:, find(I == j)); %original clustering
37 end
38
39 %Visualisation
40 %2D scatterplot of the initial clustering
41 figure;
42 plot(Z(1, :), Z(2, :), 'k.', 'MarkerSize', 15)
43 hold on
44 plot(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)

```

```

45 hold on
46 plot(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
47 hold on
48 plot(X_init_medoids{3}(1, 1), X_init_medoids{3}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
49 xlabel('PC 1', 'FontSize', 11);
50 ylabel('PC 2', 'FontSize', 11);
51 legend('Wine', 'Medoids');
52 title('2D Scatter-plot of the initial clustering', 'FontSize', 13);
53 grid on;
54
55 %3D scatterplot of the intial clustering
56 figure;
57 plot3(Z(1, :), Z(2, :), Z(3, :), 'k.', 'MarkerSize', 15)
58 hold on
59 plot3(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), X_init_medoids
    {1}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
60 hold on
61 plot3(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), X_init_medoids
    {2}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
62 hold on
63 plot3(X_init_medoids{3}(1, 1), X_init_medoids{3}(2, 1), X_init_medoids
    {3}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
64 xlabel('PC 1', 'FontSize', 11);
65 ylabel('PC 2', 'FontSize', 11);
66 zlabel('PC 3', 'FontSize', 11);
67 legend('Wine', 'Medoids');
68 title('3D Scatter-plot of the initial clustering', 'FontSize', 13);
69 grid on;
70
71 %2D scatterplot of the orginal clustering
72 figure;
73 plot(Orig{1}(1, :), Orig{1}(2, :), 'r.', 'MarkerSize', 15)
74 hold on
75 plot(Orig{2}(1, :), Orig{2}(2, :), 'b.', 'MarkerSize', 15)
76 hold on
77 plot(Orig{3}(1, :), Orig{3}(2, :), 'g.', 'MarkerSize', 15)
78 xlabel('PC 1', 'FontSize', 11);
79 ylabel('PC 2', 'FontSize', 11);
80 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3');
81 title('2D Scatter-plot of the original clustering', 'FontSize', 13);
82 grid on;
83
84 %2D scatterplot of the final clustering
85 figure;
86 plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15)
87 hold on
88 plot(Dnew{2}(1, :), Dnew{2}(2, :), 'b.', 'MarkerSize', 15)
89 hold on
90 plot(Dnew{3}(1, :), Dnew{3}(2, :), 'g.', 'MarkerSize', 15)
91 hold on
92 plot(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
93 hold on
94 plot(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
95 hold on

```

```

96 plot(X_final_medoids{3}(1, 1), X_final_medoids{3}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
97 xlabel('PC 1', 'FontSize', 11);
98 ylabel('PC 2', 'FontSize', 11);
99 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3', 'Medoids');
100 title('K-medoids (PAM) Algorithm', 'FontSize', 16);
101 subtitle('2D Scatter-plot of the final clustering', 'FontSize', 11.5);
102 grid on;
103
104 %3D Scatter-plot of the final clustering
105 figure;
106 plot3(Dnew{1}(1, :), Dnew{1}(2, :), Dnew{1}(3, :), 'r.', 'MarkerSize',
    15)
107 hold on
108 plot3(Dnew{2}(1, :), Dnew{2}(2, :), Dnew{2}(3, :), 'b.', 'MarkerSize',
    15)
109 hold on
110 plot3(Dnew{3}(1, :), Dnew{3}(2, :), Dnew{3}(3, :), 'g.', 'MarkerSize',
    15)
111 hold on
112 plot3(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1),
    X_final_medoids{1}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
113 hold on
114 plot3(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1),
    X_final_medoids{2}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
115 hold on
116 plot3(X_final_medoids{3}(1, 1), X_final_medoids{3}(2, 1),
    X_final_medoids{3}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
117 xlabel('PC 1', 'FontSize', 11);
118 ylabel('PC 2', 'FontSize', 11);
119 zlabel('PC 3', 'FontSize', 11);
120 legend('Cultivar 1', 'Cultivar 2', 'Cultivar 3', 'Medoids');
121 title('K-medoids (PAM) Algorithm', 'FontSize', 16);
122 subtitle('3D Scatter-plot of the final clustering', 'FontSize', 11.5);
123 grid on;
124
125 %Confusion Chart
126 figure;
127 cm = confusionchart(I, I_assign);
128 title('Confusion Chart');

```

### CardiacSPECT Dataset.

```

1 %% Cardiac SPECT Datas
2 clear all
3 close all
4 load CardiacSPECT.mat
5
6
7 %distance matrix: Jaccard dissimilarity index
8 [n,p]=size(X);
9
10 D = zeros(p);
11 for i = 1:p
12     for j = i+1:p
13         %Jaccard Dissimilarity index

```

```

14         n10_n01 = sum((X(:, i) == 1 & X(:, j) == 0) | (X(:, i) == 0 & X
(:, j) == 1));
15         n11 = sum(X(:, i) == 1 & X(:, j) == 1);
16         un = n10_n01 + n11;
17
18         if un == 0
19             jacDist = 0;
20         else
21             jacDist = n10_n01 / un;
22         end
23
24         % Fill in the distance matrix
25         D(i, j) = jacDist;
26         D(j, i) = jacDist; % Distance matrix is symmetric
27     end
28 end
29
30 %k_medoids
31 [I_assign, I_m, starting_medoids] = my_k_medoids(D, 2, 1.0e-100);
32
33 %PCA
34 D_cen = D - mean(D, 2);
35 [U,~,~] = svd(D_cen);
36 Z = U' * D_cen;
37
38 %Clustering
39 I_assign = I_assign - 1;
40 for j = 1:2
41     Dnew{j} = Z(:, find(I_assign == j-1)); %k-ith cluster
42     X_final_medoids{j} = Z(:, I_m(j));
43     X_init_medoids{j} = Z(:, starting_medoids(j));
44
45     Orig{j} = Z(:, find(I == j-1)); %original clustering
46 end
47
48 %matrix C
49
50
51 %Visualisation
52 %2D scatterplot of the initial clustering
53 figure;
54 plot(Z(1, :), Z(2, :), 'k.', 'MarkerSize', 15)
55 hold on
56 plot(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), 'rx', '
MarkerSize', 18, 'LineWidth', 2)
57 hold on
58 plot(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), 'rx', '
MarkerSize', 18, 'LineWidth', 2)
59 xlabel('PC 1', 'FontSize', 11);
60 ylabel('PC 2', 'FontSize', 11);
61 legend('Patients', 'Medoids');
62 title('2D Scatter-plot of the initial clustering', 'FontSize', 13);
63 grid on;
64
65 %3D scatterplot of the intial clustering
66 figure;
67 plot3(Z(1, :), Z(2, :), Z(3, :), 'k.', 'MarkerSize', 15)
68 hold on

```

```

69 plot3(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), X_init_medoids
    {1}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
70 hold on
71 plot3(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), X_init_medoids
    {2}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
72 xlabel('PC 1', 'FontSize', 11);
73 ylabel('PC 2', 'FontSize', 11);
74 zlabel('PC 3', 'FontSize', 11);
75 legend('Patients', 'Medoids');
76 title('3D Scatter-plot of the initial clustering', 'FontSize', 13);
77 grid on;
78
79 %2D Scatterplot of the original clustering
80 figure;
81 plot(Orig{1}(1, :), Orig{1}(2, :), 'r.', 'MarkerSize', 15)
82 hold on
83 plot(Orig{2}(1, :), Orig{2}(2, :), 'g.', 'MarkerSize', 15)
84 xlabel('PC 1', 'FontSize', 11);
85 ylabel('PC 2', 'FontSize', 11);
86 legend('Group A', 'Group B');
87 title('2D Scatter-plot of the original clustering', 'FontSize', 13);
88 grid on;
89
90 %2D scatterplot of the final clustering
91 figure;
92 plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15)
93 hold on
94 plot(Dnew{2}(1, :), Dnew{2}(2, :), 'g.', 'MarkerSize', 15)
95 hold on
96 plot(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
97 hold on
98 plot(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
99 xlabel('PC 1', 'FontSize', 11);
100 ylabel('PC 2', 'FontSize', 11);
101 legend('Group A', 'Group B', 'Medoids');
102 title('K-medoids (PAM) Algorithm', 'FontSize', 16);
103 subtitle('2D Scatter-plot of the final clustering', 'FontSize', 11.5);
104 grid on;
105
106 %3D Scatter-plot of the final clustering
107 figure;
108 plot3(Dnew{1}(1, :), Dnew{1}(2, :), Dnew{1}(3, :), 'r.', 'MarkerSize',
    15)
109 hold on
110 plot3(Dnew{2}(1, :), Dnew{2}(2, :), Dnew{2}(3, :), 'g.', 'MarkerSize',
    15)
111 hold on
112 plot3(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1),
    X_final_medoids{1}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
113 hold on
114 plot3(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1),
    X_final_medoids{2}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
115 xlabel('PC 1', 'FontSize', 11);
116 ylabel('PC 2', 'FontSize', 11);
117 zlabel('PC 3', 'FontSize', 11);
118 legend('Group A', 'Group B', 'Medoids');

```

```

119 title('K-medoids (PAM) Algorithm', 'FontSize', 16);
120 subtitle('3D Scatter-plot of the final clustering', 'FontSize', 11.5);
121 grid on;
122
123 %Confusion Chart
124 figure;
125 cm = confusionchart(I, I_assign);
126 title('Confusion Chart');

```

### Congressional Vote Dataset.

```

1 %% Congressional Vote Dataset
2 clear all
3 close all
4 load CongressionalVoteData.mat
5
6
7 %missing values
8 zero_col = all(X == 0, 1);
9 X = X(:, ~zero_col);
10 I = I(:, ~zero_col);
11 [n,p]=size(X);
12
13 %distance matrix
14 D = zeros(p);
15 for i = 1:p
16     for j = i+1:p
17         %field in which both representatives vote
18         Comm_vot = (X(:, i) ~= 0 & X(:, j) ~= 0);
19
20         % Compute dissimilarity index
21         num_disagr = sum(X(Comm_vot, i) ~= X(Comm_vot, j));
22         tot_vot = sum(Comm_vot);
23
24         if tot_vot > 0
25             dissim_ind = num_disagr / tot_vot;
26         else
27             % Assign neutral value for pairs with no simultaneous
28             voting record
29             dissim_ind = 0.5;
30         end
31
32         % Store dissimilarity index in distance matrix
33         D(i, j) = dissim_ind;
34         D(j, i) = dissim_ind; % Distance matrix is symmetric
35     end
36 end
37
38 %k_medoids
39 [I_assign, I_m, starting_medoids] = my_k_medoids(D, 2, 1.0e-100);
40
41 %PCA
42 D_cen = D - mean(D, 2);
43 [U,~,~] = svd(D_cen);
44 Z = U' * D_cen;

```

```

45 %Clustering
46 I_assign = I_assign - 1;
47 for j = 1:2
48     Dnew{j} = Z(:, find(I_assign == j-1)); %k-ith cluster
49     X_final_medoids{j} = Z(:, I_m(j));
50     X_init_medoids{j} = Z(:, starting_medoids(j));
51
52     Orig{j} = Z(:, find(I == j-1)); %original clustering
53 end
54
55 %Visualisation
56 %2D scatterplot of the initial clustering
57 figure;
58 plot(Z(1, :), Z(2, :), 'k.', 'MarkerSize', 15)
59 hold on
60 plot(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
61 hold on
62 plot(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
63 xlabel('PC 1', 'FontSize', 11);
64 ylabel('PC 2', 'FontSize', 11);
65 legend('Patients', 'Medoids');
66 title('2D Scatter-plot of the initial clustering', 'FontSize', 13);
67 grid on;
68
69 %3D scatterplot of the intial clustering
70 figure;
71 plot3(Z(1, :), Z(2, :), Z(3, :), 'k.', 'MarkerSize', 15)
72 hold on
73 plot3(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), X_init_medoids
    {1}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
74 hold on
75 plot3(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), X_init_medoids
    {2}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
76 xlabel('PC 1', 'FontSize', 11);
77 ylabel('PC 2', 'FontSize', 11);
78 zlabel('PC 3', 'FontSize', 11);
79 legend('Patients', 'Medoids');
80 title('3D Scatter-plot of the initial clustering', 'FontSize', 13);
81 grid on;
82
83 %2D scatterplot of the original clustering
84 figure;
85 plot(Orig{1}(1, :), Orig{1}(2, :), 'r.', 'MarkerSize', 15)
86 hold on
87 plot(Orig{2}(1, :), Orig{2}(2, :), 'g.', 'MarkerSize', 15)
88 xlabel('PC 1', 'FontSize', 11);
89 ylabel('PC 2', 'FontSize', 11);
90 legend('Cultivar 1', 'Cultivar 2');
91 title('2D Scatter-plot of the final original clustering', 'FontSize',
    13);
92 grid on;
93
94 %2D scatterplot of the final clustering
95 figure;
96 plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15)
97 hold on

```

```

98 plot(Dnew{2}(1, :), Dnew{2}(2, :), 'g.', 'MarkerSize', 15)
99 hold on
100 plot(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
101 hold on
102 plot(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
103 xlabel('PC 1', 'FontSize', 11);
104 ylabel('PC 2', 'FontSize', 11);
105 legend('Cultivar 1', 'Cultivar 2', 'Medoids');
106 title('K-medoids (PAM) Algorithm');
107 subtitle('2D Scatter-plot of the final clustering', 'FontSize', 11.5);
108 grid on;
109
110 %3D Scatter-plot of the final clustering
111 figure;
112 plot3(Dnew{1}(1, :), Dnew{1}(2, :), Dnew{1}(3, :), 'r.', 'MarkerSize',
    15)
113 hold on
114 plot3(Dnew{2}(1, :), Dnew{2}(2, :), Dnew{2}(3, :), 'g.', 'MarkerSize',
    15)
115 hold on
116 plot3(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1),
    X_final_medoids{1}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
117 hold on
118 plot3(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1),
    X_final_medoids{2}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
119 xlabel('PC 1', 'FontSize', 11);
120 ylabel('PC 2', 'FontSize', 11);
121 zlabel('PC 3', 'FontSize', 11);
122 legend('Cultivar 1', 'Cultivar 2', 'Medoids');
123 title('K-medoids (PAM) Algorithm');
124 subtitle('3D Scatter-plot of the final clustering', 'FontSize', 11.5);
125 grid on;
126
127 %Confusion Chart
128 figure;
129 cm = confusionchart(I, I_assign);
130 title('Confusion Chart');

```

## Biopsy Dataset.

```

1
2 %% Biopsy Dataset
3 clear all
4 close all
5 load BiopsyData.mat
6
7 %missing values
8 NaN_col = any(isnan(X), 1);
9 X = X(:, ~NaN_col);
10
11 %distance matrix
12 [n, p] = size(X);
13
14 D = zeros(p);

```



```

15 for i = 1:p
16     for j = i:p
17         if (i == j)
18             D(i,j) = 0;
19         else
20             D(i,j) = norm(X(:, i) - X(:, j), 2);
21             %TRY ALSO NORM 1 and NORM 'inf'
22             %compare also the iteration number
23         end
24     end
25 %     end
26 end
27 D = D+D';
28
29 %k-medoids
30 [I_assign, I_m, starting_medoids] = my_k_medoids(D, 2, 1.0e-100);
31
32 %PCA
33 D_cen = D - mean(D, 2);
34 [U,~,~] = svd(D_cen);
35 Z = U' * D_cen;
36
37 %clustering
38 for j = 1:2
39     Dnew{j} = Z(:, find(I_assign == j)); %k-ith cluster
40     X_final_medoids{j} = Z(:, I_m(j));
41     X_init_medoids{j} = Z(:, starting_medoids(j));
42 end
43
44 %visualisation
45 %2D scatterplot of the initial clustering
46 figure;
47 plot(Z(1, :), Z(2, :), 'k.', 'MarkerSize', 15)
48 hold on
49 plot(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
50 hold on
51 plot(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), 'rx', '
    MarkerSize', 18, 'LineWidth', 2)
52 xlabel('PC 1', 'FontSize', 11);
53 ylabel('PC 2', 'FontSize', 11);
54 legend('Biopsy data', 'Medoids');
55 title('2D Scatter-plot of the initial clustering', 'FontSize', 13);
56 grid on;
57
58 %3D scatterplot of the intial clustering
59 figure;
60 plot3(Z(1, :), Z(2, :), Z(3, :), 'k.', 'MarkerSize', 15)
61 hold on
62 plot3(X_init_medoids{1}(1, 1), X_init_medoids{1}(2, 1), X_init_medoids
    {1}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
63 hold on
64 plot3(X_init_medoids{2}(1, 1), X_init_medoids{2}(2, 1), X_init_medoids
    {2}(3, 1), 'rx', 'MarkerSize', 18, 'LineWidth', 2)
65 xlabel('PC 1', 'FontSize', 11);
66 ylabel('PC 2', 'FontSize', 11);
67 zlabel('PC 3', 'FontSize', 11);
68 legend('Biopsy data', 'Medoids');

```

```

69 title('3D Scatter-plot of the initial clustering', 'FontSize', 13);
70 grid on;
71
72 %2D scatterplot of the final clustering
73 figure;
74 plot(Dnew{1}(1, :), Dnew{1}(2, :), 'r.', 'MarkerSize', 15)
75 hold on
76 plot(Dnew{2}(1, :), Dnew{2}(2, :), 'g.', 'MarkerSize', 15)
77 hold on
78 plot(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
79 hold on
80 plot(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1), 'kx', '
    MarkerSize', 18, 'LineWidth', 2)
81 xlabel('PC 1', 'FontSize', 11);
82 ylabel('PC 2', 'FontSize', 11);
83 legend('Benign', 'Malignant', 'Medoids');
84 title('K-medoids (PAM) Algorithm', 'FontSize', 16);
85 subtitle('2D Scatter-plot of the final clustering', 'FontSize', 11.5);
86 grid on;
87
88 %3D Scatter-plot of the final clustering
89 figure;
90 plot3(Dnew{1}(1, :), Dnew{1}(2, :), Dnew{1}(3, :), 'r.', 'MarkerSize',
    15)
91 hold on
92 plot3(Dnew{2}(1, :), Dnew{2}(2, :), Dnew{2}(3, :), 'g.', 'MarkerSize',
    15)
93 hold on
94 plot3(X_final_medoids{1}(1, 1), X_final_medoids{1}(2, 1),
    X_final_medoids{1}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
95 hold on
96 plot3(X_final_medoids{2}(1, 1), X_final_medoids{2}(2, 1),
    X_final_medoids{2}(3, 1), 'kx', 'MarkerSize', 18, 'LineWidth', 2)
97 xlabel('PC 1', 'FontSize', 11);
98 ylabel('PC 2', 'FontSize', 11);
99 zlabel('PC 3', 'FontSize', 11);
100 legend('Benign', 'Malignant', 'Medoids');
101 title('K-medoids (PAM) Algorithm', 'FontSize', 16);
102 subtitle('3D Scatter-plot of the final clustering', 'FontSize', 11.5);
103 grid on;

```